

Subject Name : Machine Learning

Unit No:5

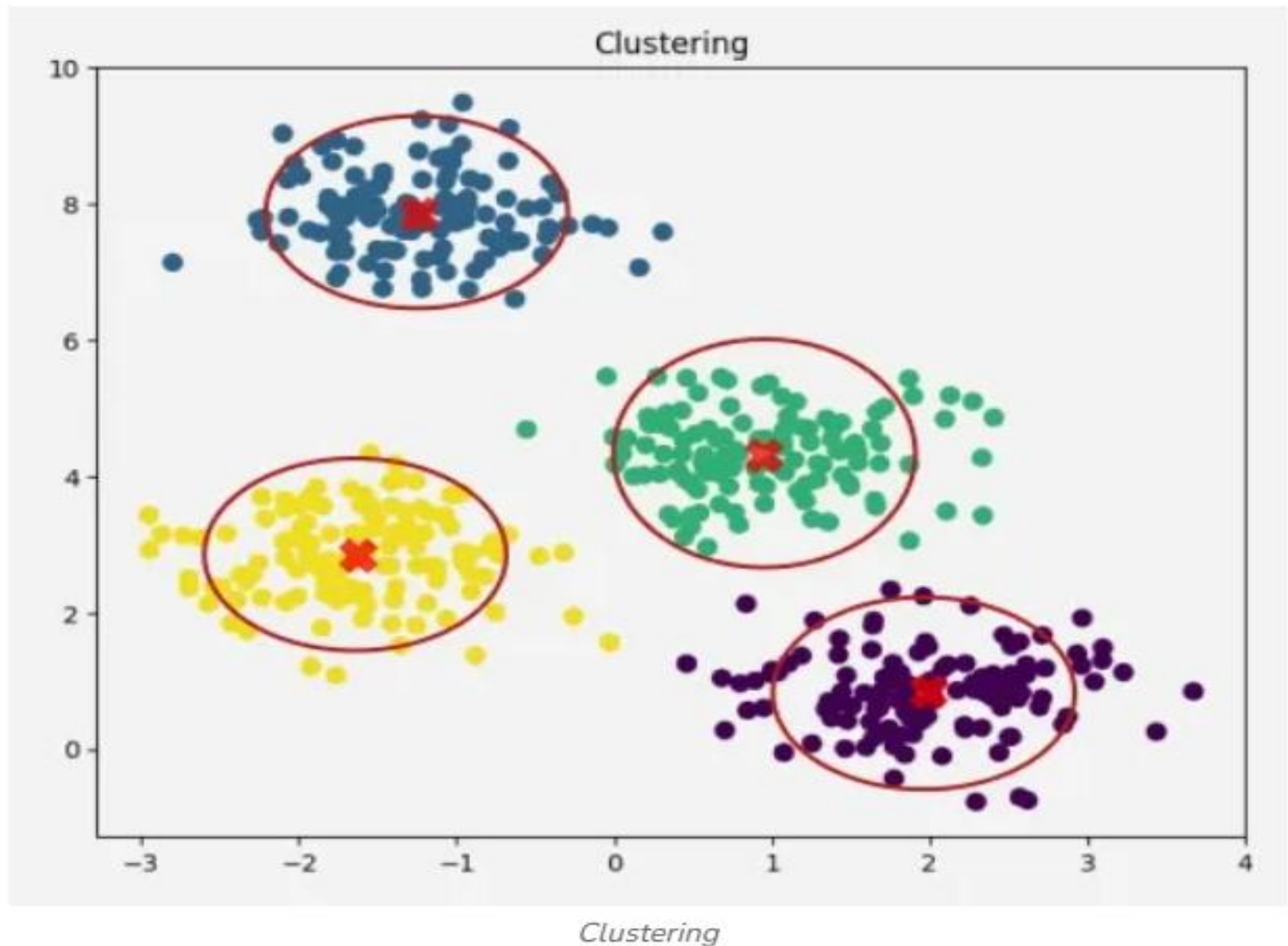
Unit Name : Unsupervised Learning

Faculty Name : Dr. Ritu Jain

Clustering

- Clustering is a Unsupervised learning technique used to group unlabeled data points into clusters based on inherent similarities and hidden patterns.
- It enables discovery of natural groupings within data without predefined categories, commonly used for customer segmentation, data organization, and anomaly detection.
- It helps discover hidden patterns or natural groupings in datasets by placing similar data points into the same cluster.
- Data points are assigned to clusters based on similarity or distance measures. Uses Euclidean distance, cosine similarity or other metrics depending on data type and clustering method.





L

Types of Clustering

- ✓ 1. **Hard Clustering:** Hard clustering assigns each data point to exactly one cluster.
- ✓ • A data point cannot belong to multiple clusters, making the grouping clear and easy to interpret.
- ✓ • No overlap between clusters
- ✓ • **Example:**
- ✓ • If customers are divided into two clusters, each customer belongs completely to either Cluster 1 or Cluster 2.
- ✓ • A customer cannot belong to both clusters at the same time.



Soft Clustering

- Soft clustering allows a data point to belong to multiple clusters with different probabilities.
- Instead of assigning a strict cluster, it gives a degree of membership to each cluster.
- **Example**
 - A data point may belong 70% to Cluster 1 and 30% to Cluster 2, indicating that it shares characteristics with both groups.
- **Use Cases**
 - **Overlapping class boundaries:** Useful when data points cannot be clearly separated into distinct groups.
 - **Customer personas:** Helps represent customers who share traits with multiple behavioral groups.
 - **Medical diagnosis:** Patients may show symptoms related to multiple conditions.



Clustering Distance Measures

- ✓ Clustering is a fundamental concept in data analysis and machine learning, where the goal is to group similar data points into clusters based on their characteristics.
- ✓ **One of the most critical aspects of clustering is the choice of distance measure, which determines how similar or dissimilar two data points are.**
- ✓ Distance measures are mathematical functions that determine how similar or different two data points are.
- ✓ **The choice of distance measure can significantly impact the clustering results, as it influences the shape and structure of the clusters.**



Clustering Distance Measures

- ✓ The choice of distance measure significantly *impacts the quality of the clusters formed and the insights derived from them.*
- ✓ A well-chosen distance measure can lead to meaningful clusters that reveal hidden patterns in the data, while a poorly chosen measure can result in clusters that are misleading or irrelevant.
- ✓ Distance measurements specify how similarity between data points is assessed which makes them essential for grouping.
- ✓ The performance of the clustering method, and its result can be strongly impacted by the distance measure selection.
- ✓ It affects the formation of clusters and may have an impact on the validity and interpretability of the clusters.



Common Distance Measures

- Here are some of the most commonly used distance measures in clustering:

✓ 1. Euclidean Distance

✓ 2. Manhattan Distance

✓ 3. Cosine Similarity



Euclidean Distance

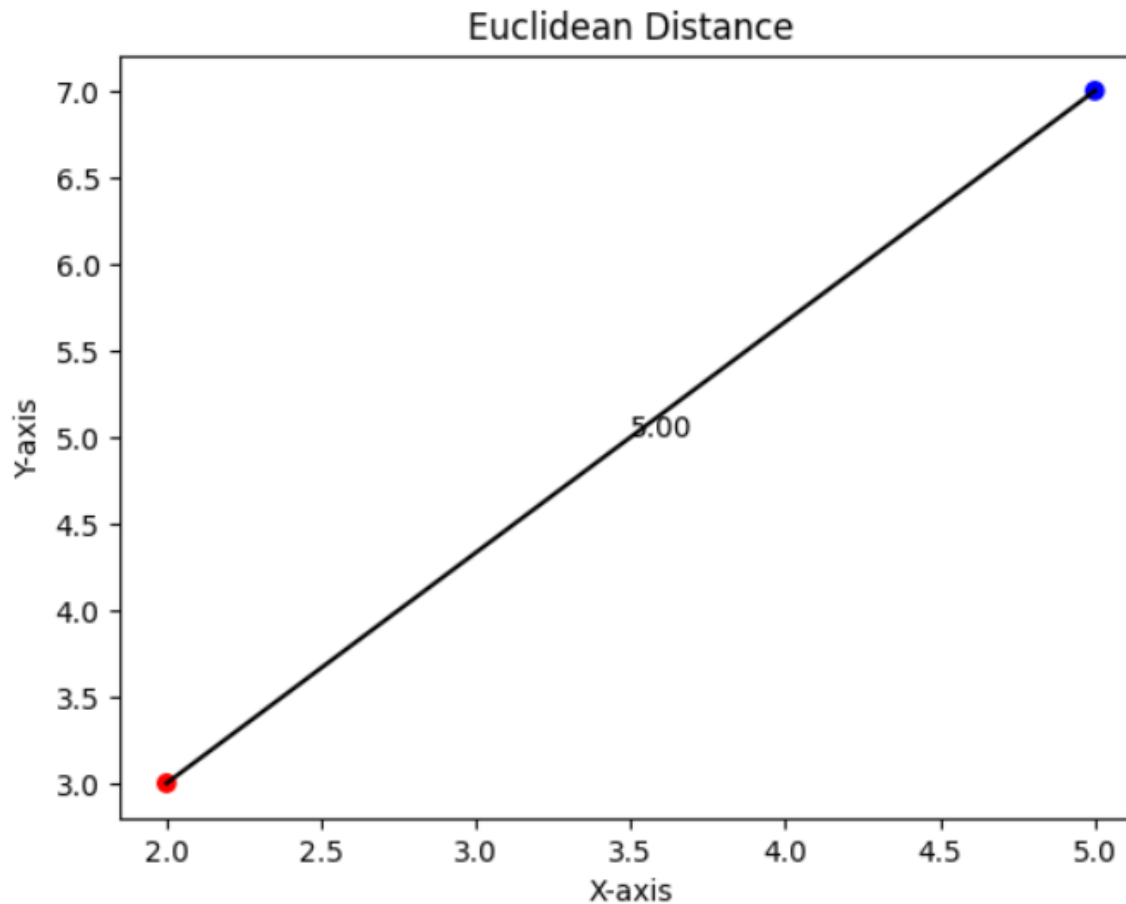
- The Euclidean distance is the most widely used distance measure in clustering. It calculates the straight-line distance between two points in n-dimensional space. The formula for Euclidean distance is:

- $$d(p, q) = \sqrt{\sum_{i=1}^n (p_i - q_i)^2}$$

- where,
- p and q are two data points
- and n is the number of dimensions.

- ✓ **Best Use:** Dense, low-dimensional data, or when the absolute magnitude of differences is crucial (e.g., K-Means clustering, KNN).





point1 = [2, 3], point2 = [5, 7]
Euclidean Distance: 5.0



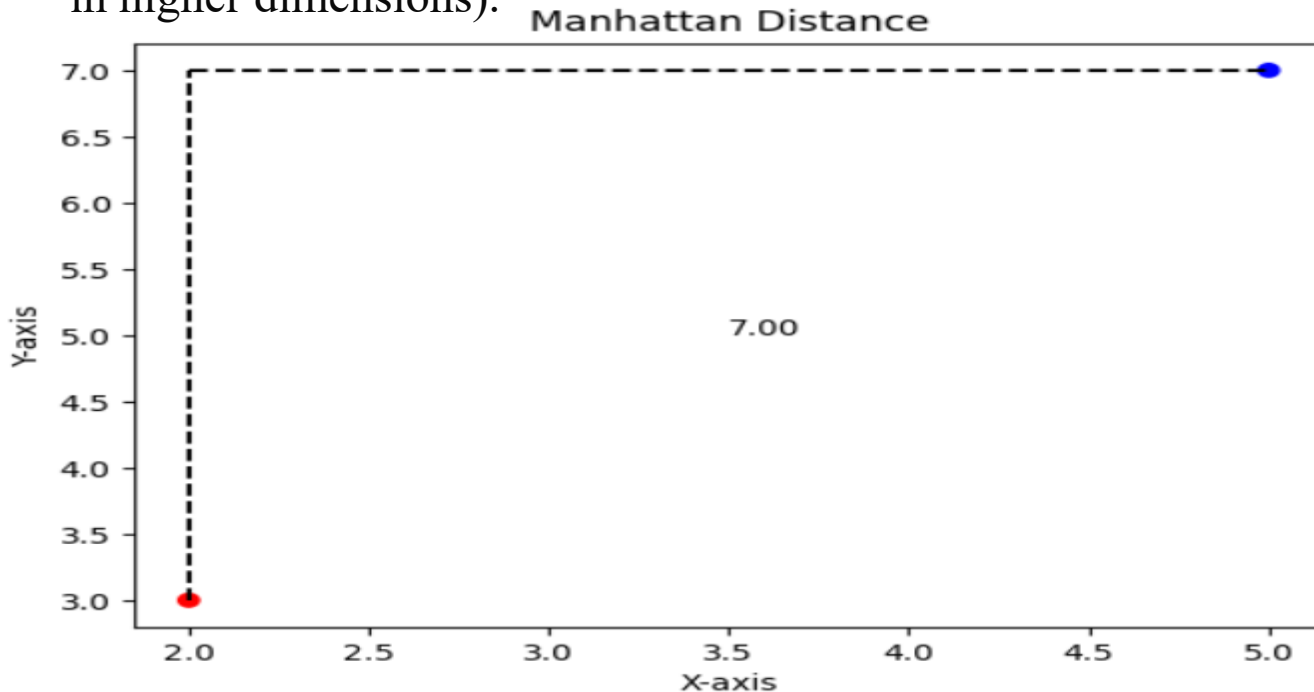
Manhattan Distance

- The Manhattan distance, is the total of the absolute differences between their Cartesian coordinates, sometimes referred to as the L1 distance or city block distance.
- Envision maneuvering across a city grid in which your only directions are horizontal and vertical.
- The Manhattan distance, which computes the total distance traveled along each dimension to reach a different data point represents this movement.
- When it comes to categorical data this metric is more effective than Euclidean distance since it is less susceptible to outliers. The formula is:
 - $d(p, q) = \sum_{i=1}^n |p_i - q_i|$



Manhattan Distance

- **Best Use:** High-dimensional datasets, sparse data, or scenarios involving grid movement (e.g., Manhattan distance is often preferred over Euclidean in higher dimensions).



The two points are represented by the red and blue points in the plot. The grid-line-based method used to determine the Manhattan distance is depicted by the dashed black lines.

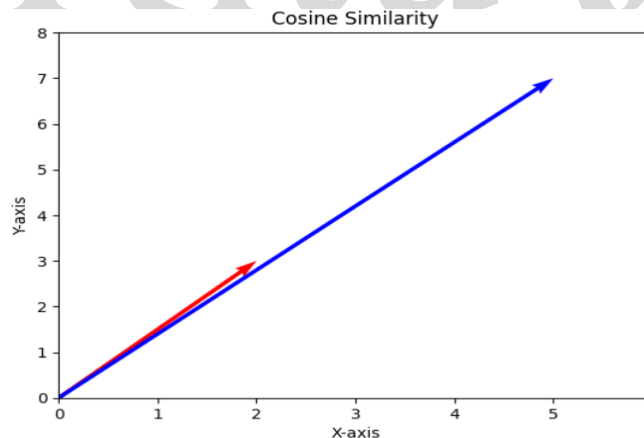
Cosine Similarity

- Instead than concentrating on the exact distance between data points, cosine similarity measure looks at their orientation.
- It calculates the cosine of the angle between two data points, with a higher cosine value indicating greater similarity.
- This measure is often used for text data analysis, where the order of features (words in a sentence) might not be as crucial as their presence.
- It is used to determine how similar the vectors are, irrespective of their magnitude.
- $$\text{similarity}(A, B) = \frac{A \cdot B}{\|A\| \|B\|}$$



Cosine Similarity

- **Focus:** Orientation (direction) rather than magnitude. It calculates how "similar" the direction is, ignoring how far apart they are.
- **Best Use:** High-dimensional data where magnitude doesn't matter (e.g., Text mining/NLP to compare documents, recommendation systems).
- **Range:** -1 (opposite directions) to 1 (same direction). 0 means orthogonal.



Cosine Similarity: 0.9994801143396996



D Y PATIL
UNIVERSITY
NAVI MUMBAI

Differentiate between euclidean distance, manhattan distance, and cosine similarity

Feature	Euclidean Distance	Manhattan Distance	Cosine Similarity
Type	Distance metric	Distance metric	Similarity measure
Formula Basis	Square root of squared differences	Sum of absolute differences	Angle between vectors
Sensitivity to Scale	High	Moderate	Low
Effect of Magnitude	Important	Important	Ignored
Range	0 to ∞	0 to ∞	-1 to 1
Use Cases	Clustering, KNN	High-dimensional data	Text similarity, NLP
Outlier Sensitivity	High	Lower	Not directly affected



When to Use Which Distance measure?

- Use Cosine Similarity when magnitude doesn't matter (e.g., two documents are similar, even if one is much longer than the other).
- Use Euclidean Distance when the absolute distance between points matters (e.g., physical distances, sensor data).
- Use Manhattan Distance if your data has significant outliers or high-dimensional characteristics



Quality: What Is Good Clustering?

- A good clustering method will produce high quality clusters
- high intra-class similarity: cohesive within clusters
- low inter-class similarity: distinctive between clusters
- The quality of a clustering method depends on
 - the similarity measure used by the method
 - its implementation, and
 - Its ability to discover some or all of the hidden patterns



Common Clustering Algorithms

- **K-Means:** Partitions data into clusters by minimizing the distance between points and their cluster centroid.
- **Hierarchical Clustering:** Creates a tree structure of clusters through agglomerative (merging) or divisive (splitting) approaches.
- **DBSCAN (Density-Based):** Identifies clusters based on dense areas of data points, effective for identifying noise.



K means Clustering

- K-means clustering is a simple and elegant approach for partitioning a data set into K distinct, non-overlapping clusters.
- K-means clustering is the most commonly used unsupervised machine learning algorithm for partitioning a given dataset into a set of k clusters, where k represents the number of groups pre-specified by the user.
- In k-means clustering, each cluster is represented by its center or centroid which corresponds to the mean of points assigned to the cluster.
- The basic idea behind k-means clustering is to define clusters and their centroids such that the total intra-cluster variation is minimized.



-
- K-means is popular because of its speed and scalability.
 - K-means algorithm defines the total within-cluster variation as the sum of squared Euclidean distances between items and the corresponding centroid.
 - K is a hyperparameter of the algorithm



Algorithm: K means Clustering

Iterative algo

k-means algorithm can be summarized as follows:

- ✓ 1. Specify the number of clusters (k) to be created (to be specified by users).
- ✓ 2. Select k data points randomly from the dataset as the initial cluster centers or means.
- ✓ 3. Assign each data point to their closest centroid, based on the euclidean distance between a data point and its centroid.
- ✓ 4. For each of the k clusters update cluster centroid by calculating the new mean values of all the data points in the cluster.
- ✓ 5. Iteratively minimize the total within the sum of squares: iterate steps 3 and 4 until the cluster assignments stop changing, centroid stop changing or the maximum number of iterations is reached.



Stopping Criteria for K-Means Clustering

- There are essentially three stopping criteria that can be adopted to stop the K-means algorithm:

- ✓ 1. Centroids of newly formed clusters do not change

- ✓ 2. Points remain in the same cluster

- ✓ 3. Maximum number of iterations is reached

- ✓ • We can stop the algorithm if the centroids of newly formed clusters are not changing. Even after multiple iterations, if we are getting the same centroids for all the clusters, we can say that the algorithm is not learning any new pattern, and it is a sign to stop the training.

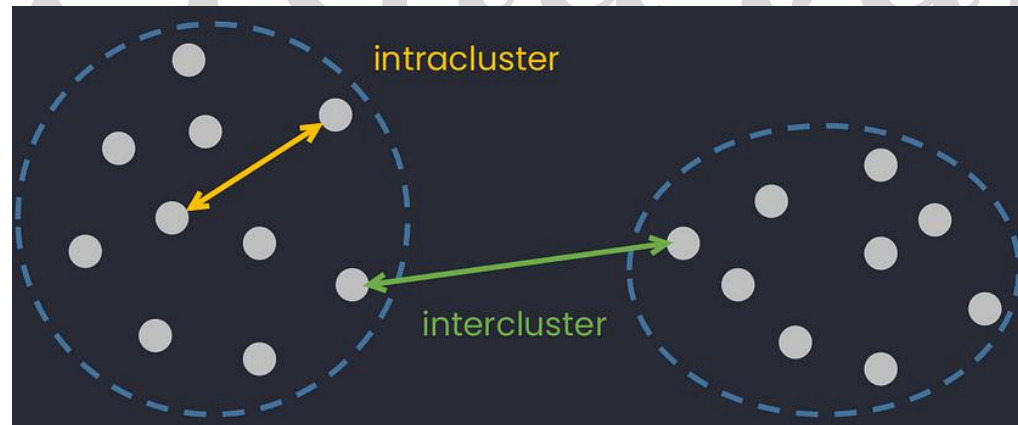
- ✓ • Another clear sign that we should stop the training process is if the points remain in the same cluster even after training the algorithm for multiple iterations.

- ✓ • Finally, we can stop the training if the maximum number of iterations is reached. Suppose we have set the number of iterations as 100. The process will repeat for 100 iterations before stopping



Main points in k means clustering

- ✓ **Intercluster distance should be high:** The distance between observations in two clusters should be High.
- ✓ **Intracluster Distance should be Very Less:** The distance of observation within the cluster should be very less.



WCSS (Within-Cluster Sum of Squares) or Inertia

- In K-Means clustering, WCSS (Within-Cluster Sum of Squares) is a metric used to evaluate how compact or "tight" your clusters are. It is also known as **inertia**.
- **How to Calculate WCSS:** To calculate the total WCSS for a model, you follow these three main steps:
 1. Find Squared Distance: For every data point, calculate the squared Euclidean distance to its assigned cluster centroid.
 2. Sum Within Clusters: Sum these squared distances for all points within a specific cluster.
 3. Total Sum: Add the sums from every cluster together to get the final WCSS value.
- The mathematical formula is:
$$WCSS = \sum_{i=1}^k \sum_{j \in C_i} ||x_j - \mu_i||^2$$
- Where k is the number of clusters, x_j is a data point in cluster , and μ_i is the centroid of that cluster.



k-means clustering: Within-Cluster Sum of Squares (WCSS)

- In k-means clustering, WCSS (Within-Cluster Sum of Squares) is the primary metric used to evaluate how compact and well-defined your clusters are. It represents the sum of the squared distances between each data point and the centroid of its assigned cluster.
- **The Role of WCSS**
 - The k-means algorithm is an iterative process with a singular goal: to minimize WCSS.
 - Low WCSS: Indicates that data points are very close to their respective centroids, resulting in tight, dense clusters.
 - High WCSS: Indicates that data points are spread far from their centroids, suggesting the clusters are not well-formed or the number of clusters is insufficient.

WCSS measures the compactness (i.e., goodness) of the resulting clusters and we want it to be as small as possible

$$WCSS(k) = \sum_{j=1}^k \sum_{\mathbf{x}_i \in \text{cluster } j} \|\mathbf{x}_i - \bar{\mathbf{x}}_j\|^2,$$

where $\bar{\mathbf{x}}_j$ is the sample mean in cluster j



D Y PATIL
UNIVERSITY
NAVI MUMBAI

-
- The parameters that minimize the cost function are learned through an iterative process of assigning data points to clusters and then moving the clusters. A restriction for the k-means algorithm is that the dataset should be continuous.

Dr. Ritu Jain



-
- Apply K(=2)-Means algorithm over the data (185, 72), (170, 56), (168, 60), (179, 68), (182, 72), (188, 77) up to two iterations and show the clusters. Initially choose first two objects as initial centroids.

Dr. Ritu Jain



- Given, number of clusters to be created (K) = 2 say c_1 and c_2 , number of iterations = 2.

The given data points can be represented in tabular form as:

Instance	X	Y
1	185	72
2	170	56
3	168	60
4	179	68
5	182	72
6	188	77

- First two objects as initial centroids:

Centroid for first cluster $c_1 = (185, 72)$

Centroid for second cluster $c_2 = (170, 56)$



Iteration 1: Now calculating similarity by using *Euclidean distance* measure as: **Centroid for first cluster c1 = (185, 72)** **Centroid for second cluster c2 = (170, 56)**

$$d(c1, 3) = \sqrt{(185 - 168)^2 + (72 - 60)^2} = \sqrt{(17)^2 + (12)^2} = \sqrt{289 + 144} = \sqrt{433}$$

$$d(c2, 3) = \sqrt{(170 - 168)^2 + (56 - 60)^2} = \sqrt{(2)^2 + (-4)^2} = \sqrt{4 + 16} = \sqrt{20}$$

Here, $d(c2, 3) < d(c1, 3)$

So, data point 3 belongs to c2.

$$d(c1, 4) = \sqrt{(185 - 179)^2 + (72 - 68)^2} = \sqrt{(6)^2 + (4)^2} = \sqrt{36 + 16} = \sqrt{52}$$

$$d(c2, 4) = \sqrt{(170 - 179)^2 + (56 - 68)^2} = \sqrt{(-9)^2 + (-12)^2} = \sqrt{81 + 144} = \sqrt{225}$$

Here, $d(c1, 4) < d(c2, 4)$

So, data point 4 belongs to c1.

$$d(c1, 5) = \sqrt{(185 - 182)^2 + (72 - 72)^2} = \sqrt{(3)^2 + (0)^2} = \sqrt{9}$$

$$d(c2, 5) = \sqrt{(170 - 182)^2 + (56 - 72)^2} = \sqrt{(-12)^2 + (-16)^2} = \sqrt{144 + 256} = \sqrt{400}$$

Here, $d(c1, 5) < d(c2, 5)$

So, data point 5 belongs to c1.

$$d(c1, 6) = \sqrt{(185 - 188)^2 + (72 - 77)^2} = \sqrt{(-3)^2 + (-5)^2} = \sqrt{9 + 25} = \sqrt{34}$$

$$d(c2, 6) = \sqrt{(170 - 188)^2 + (56 - 77)^2} = \sqrt{(-18)^2 + (-21)^2} = \sqrt{324 + 441} = \sqrt{765}$$

Here, $d(c1, 6) < d(c2, 6)$

So, data point 6 belongs to c1.

Instance	X	Y
1	185	72
2	170	56
3	168	60
4	179	68
5	182	72
6	188	77

Instance	X	Y	Distance(C1)	Distance(C2)	Cluster
1	185	72			c1
2	170	56			c2
3	168	60	$\sqrt{433}$	$\sqrt{20}$	c2
4	179	68	$\sqrt{52}$	$\sqrt{225}$	c1
5	182	72	$\sqrt{9}$	$\sqrt{400}$	c1
6	188	77	$\sqrt{34}$	$\sqrt{765}$	c1



D Y PATIL
UNIVERSITY
NAVI MUMBAI

- Representing above information in tabular form: Distance of each data points from cluster centroids

Instance	X	Y	Distance(C1)	Distance(C2)	Cluster
1	185	72			c1
2	170	56			c2
3	168	60	$\sqrt{433}$	$\sqrt{20}$	c2
4	179	68	$\sqrt{52}$	$\sqrt{225}$	c1
5	182	72	$\sqrt{9}$	$\sqrt{400}$	c1
6	188	77	$\sqrt{34}$	$\sqrt{765}$	c1

- The resulting cluster after first iteration is:



-
- **Iteration 2:** Now calculating centroid for each cluster: Calculating centroid as mean of data points

Centroid for first cluster $c1 = \left(\frac{185+179+182+188}{4}, \frac{72+68+72+77}{4} \right) = (183.5, 72.25)$

Centroid for second cluster $c2 = \left(\frac{170+168}{2}, \frac{56+60}{2} \right) = (169, 58)$

Now, again calculating similarity:

$$d(c1, 1) = \sqrt{(183.5 - 185)^2 + (72.25 - 72)^2} = 1.5207$$

$$d(c2, 1) = \sqrt{(169 - 185)^2 + (58 - 72)^2} = 21.2603$$

Here, $d(c1, 1) < d(c2, 1)$

So, data point 1 belongs to c1.

$$d(c1, 2) = \sqrt{(183.5 - 170)^2 + (72.25 - 56)^2} = 21.1261$$

$$d(c2, 2) = \sqrt{(169 - 170)^2 + (58 - 56)^2} = 2.2361$$

Here, $d(c2, 2) < d(c1, 2)$

So, data point 2 belongs to c2.

$$d(c1, 3) = \sqrt{(183.5 - 168)^2 + (72.25 - 60)^2} = 19.7563$$

$$d(c2, 3) = \sqrt{(169 - 168)^2 + (58 - 60)^2} = 2.2361$$

Here, $d(c2, 3) < d(c1, 3)$

So, data point 3 belongs to c2.

$$d(c1, 4) = \sqrt{(183.5 - 179)^2 + (72.25 - 68)^2} = 6.1897$$

$$d(c2, 4) = \sqrt{(169 - 179)^2 + (58 - 68)^2} = 14.1421$$

Here, $d(c1, 4) < d(c2, 4)$

So, data point 4 belongs to c1.

$$d(c1, 5) = \sqrt{(183.5 - 182)^2 + (72.25 - 72)^2} = 1.5207$$

$$d(c2, 5) = \sqrt{(169 - 182)^2 + (58 - 72)^2} = 19.1050$$

Here, $d(c1, 5) < d(c2, 5)$

So, data point 5 belongs to c1.

$$d(c1, 6) = \sqrt{(183.5 - 188)^2 + (72.25 - 77)^2} = 6.5431$$

$$d(c2, 6) = \sqrt{(169 - 188)^2 + (58 - 77)^2} = 26.8701$$

Here, $d(c1, 6) < d(c2, 6)$

So, data point 6 belongs to c1.

Instance	X	Y	Distance(C1)	Distance(C2)	Cluster
1	185	72	1.5207	21.2603	c1
2	170	56	21.1261	2.2361	c2
3	168	60	19.7563	2.2361	c2
4	179	68	6.1897	14.1421	c1
5	182	72	1.5207	19.105	c1
6	188	77	6.5431	26.8701	c1

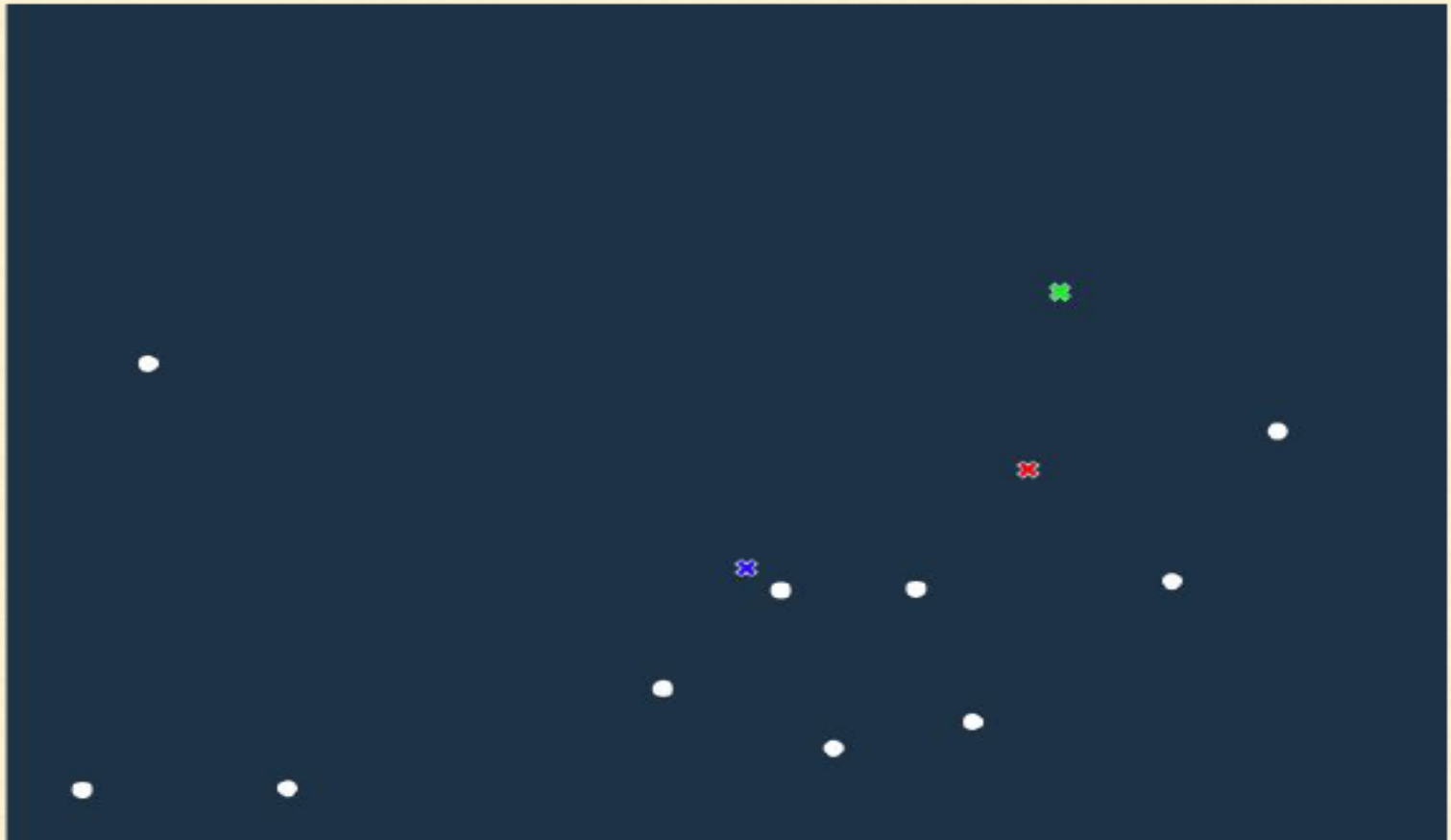


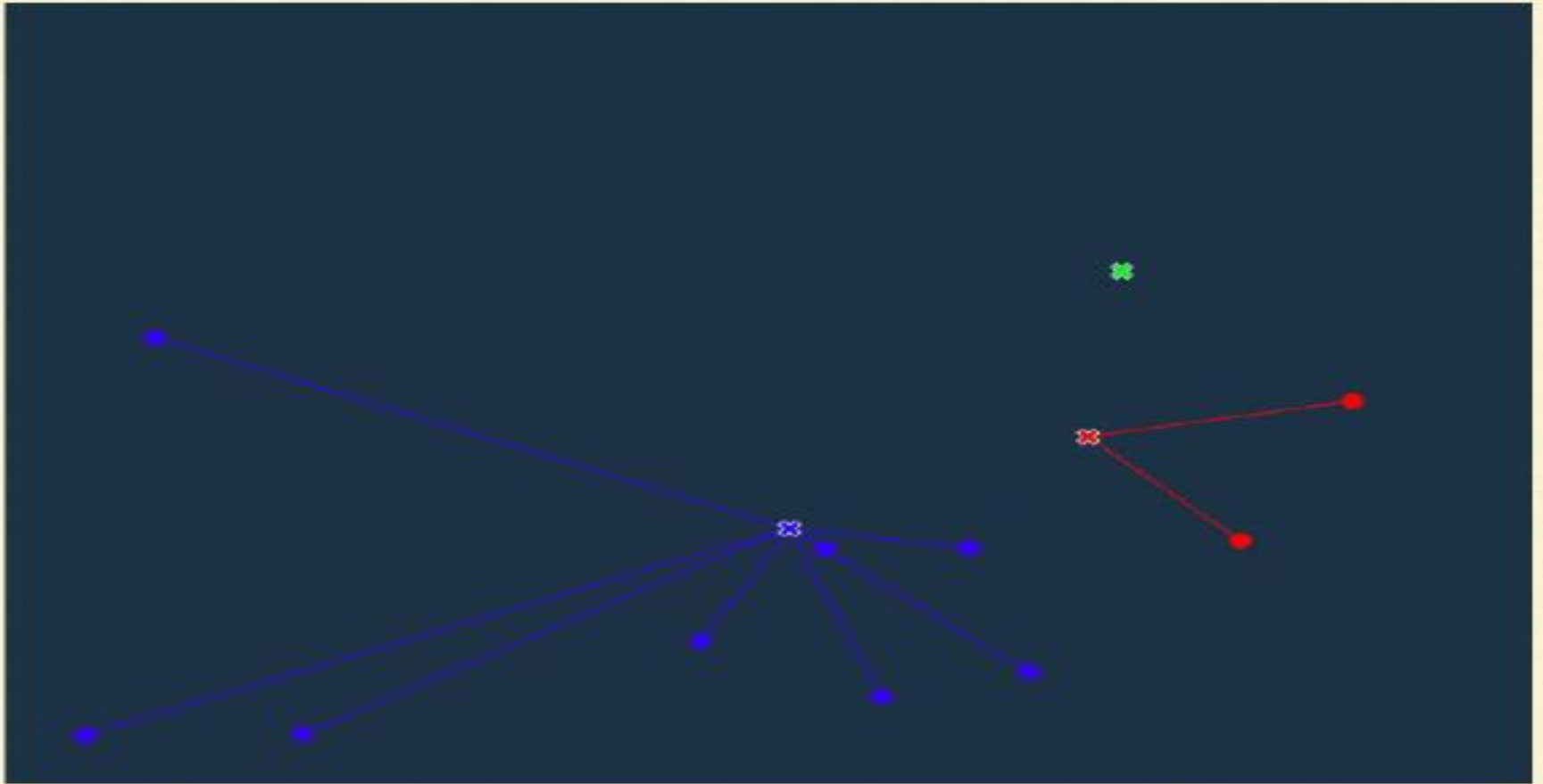
- Representing above information in tabular form.

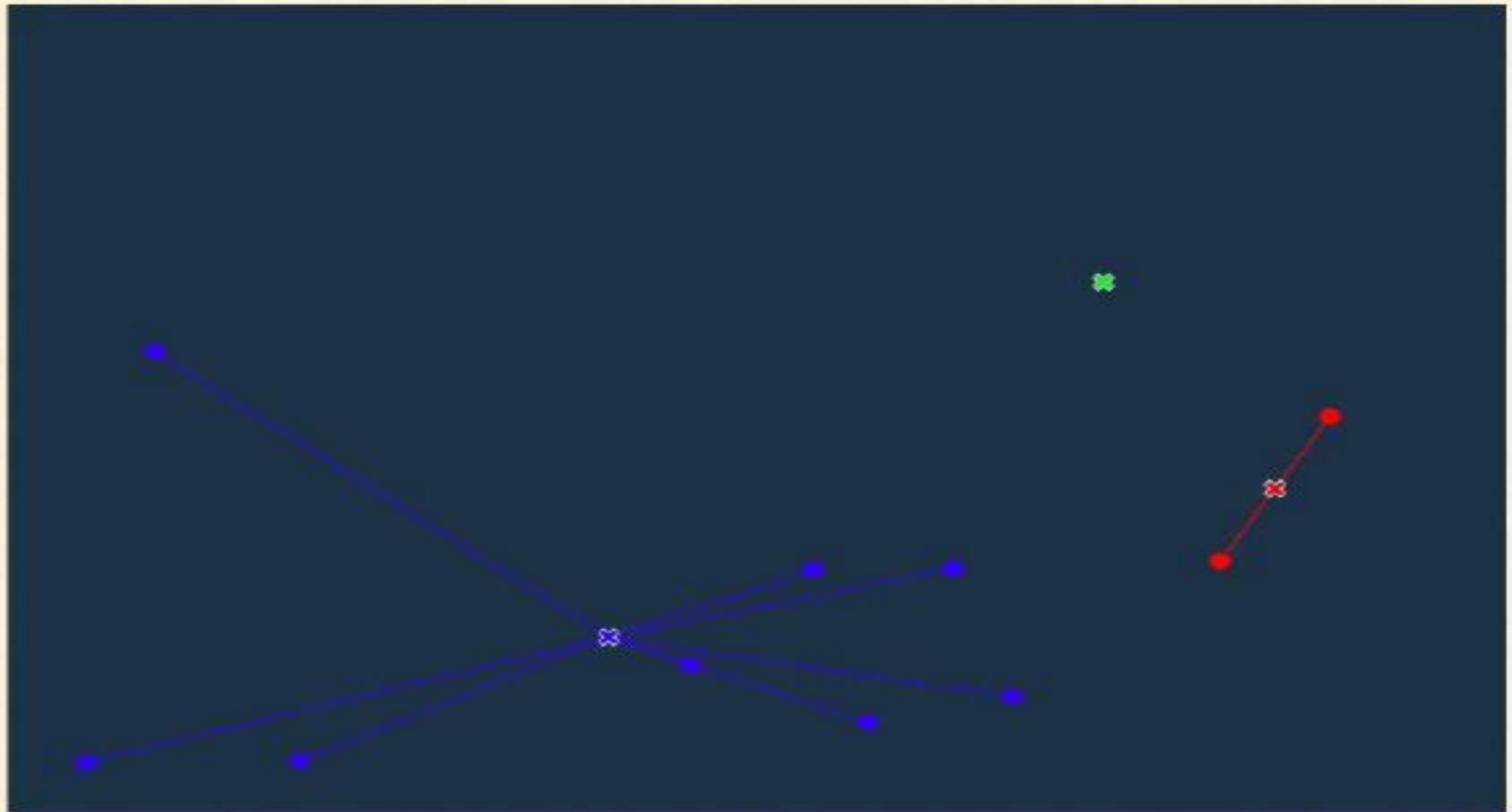
Instance	X	Y	Distance(C1)	Distance(C2)	Cluster
1	185	72	1.5207	21.2603	c1
2	170	56	21.1261	2.2361	c2
3	168	60	19.7563	2.2361	c2
4	179	68	6.1897	14.1421	c1
5	182	72	1.5207	19.105	c1
6	188	77	6.5431	26.8701	c1

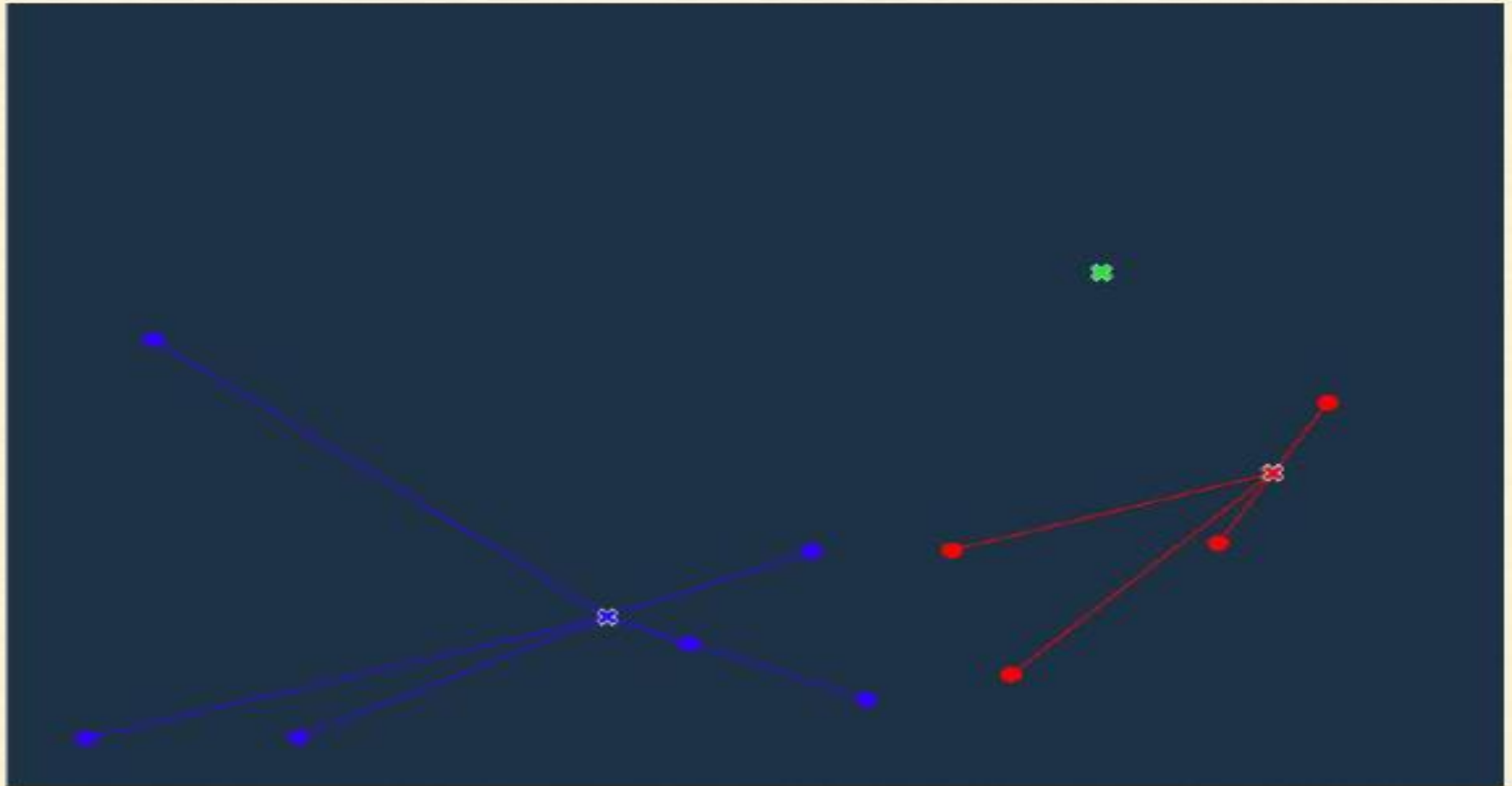
- As we have already completed two iteration as asked by our question, the numerical ends here.
- Since, the clustering doesn't change after second iteration, so terminate the iteration even if question doesn't say so.

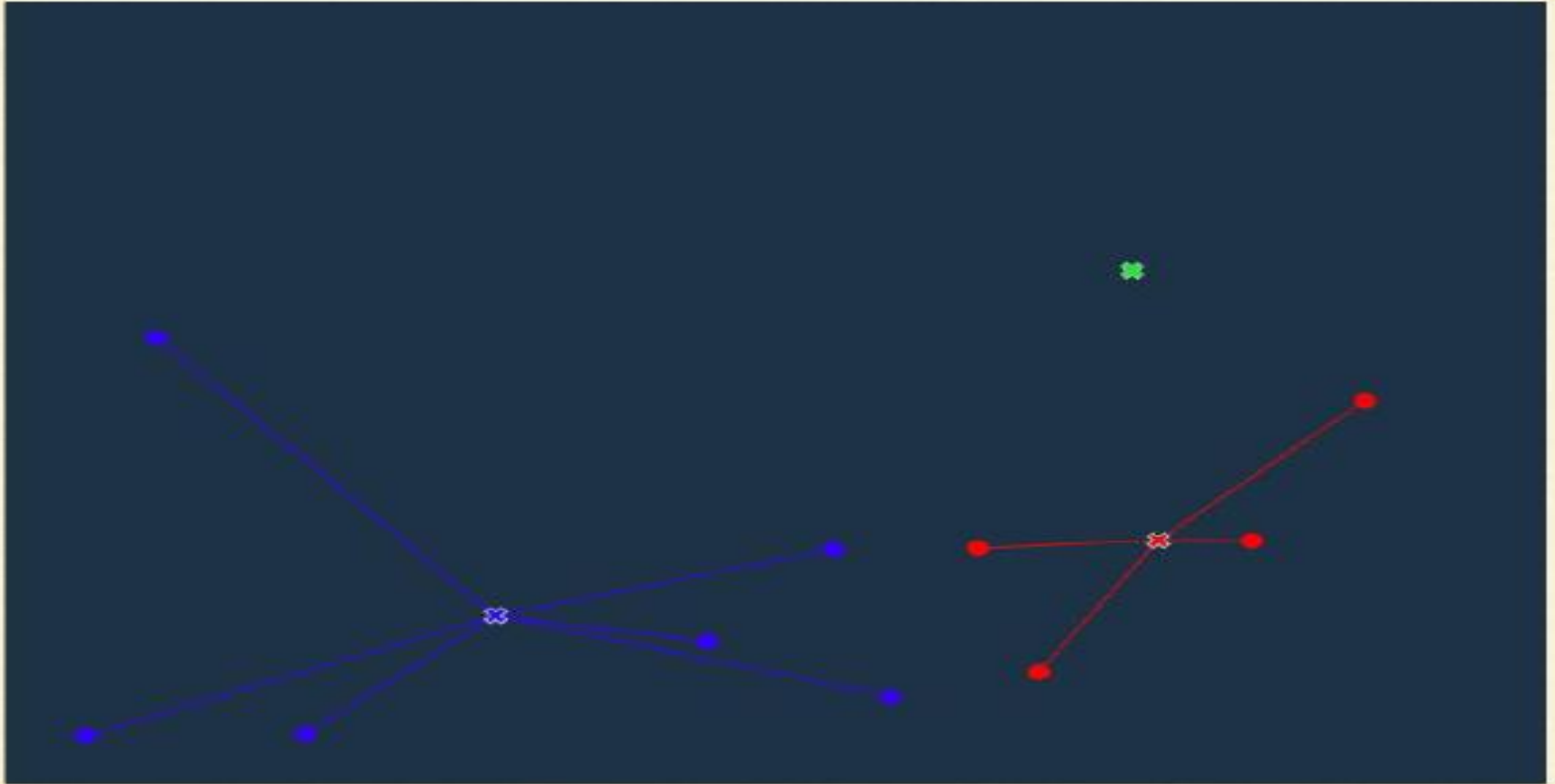
Initialize - 10 samples and 3 centroids

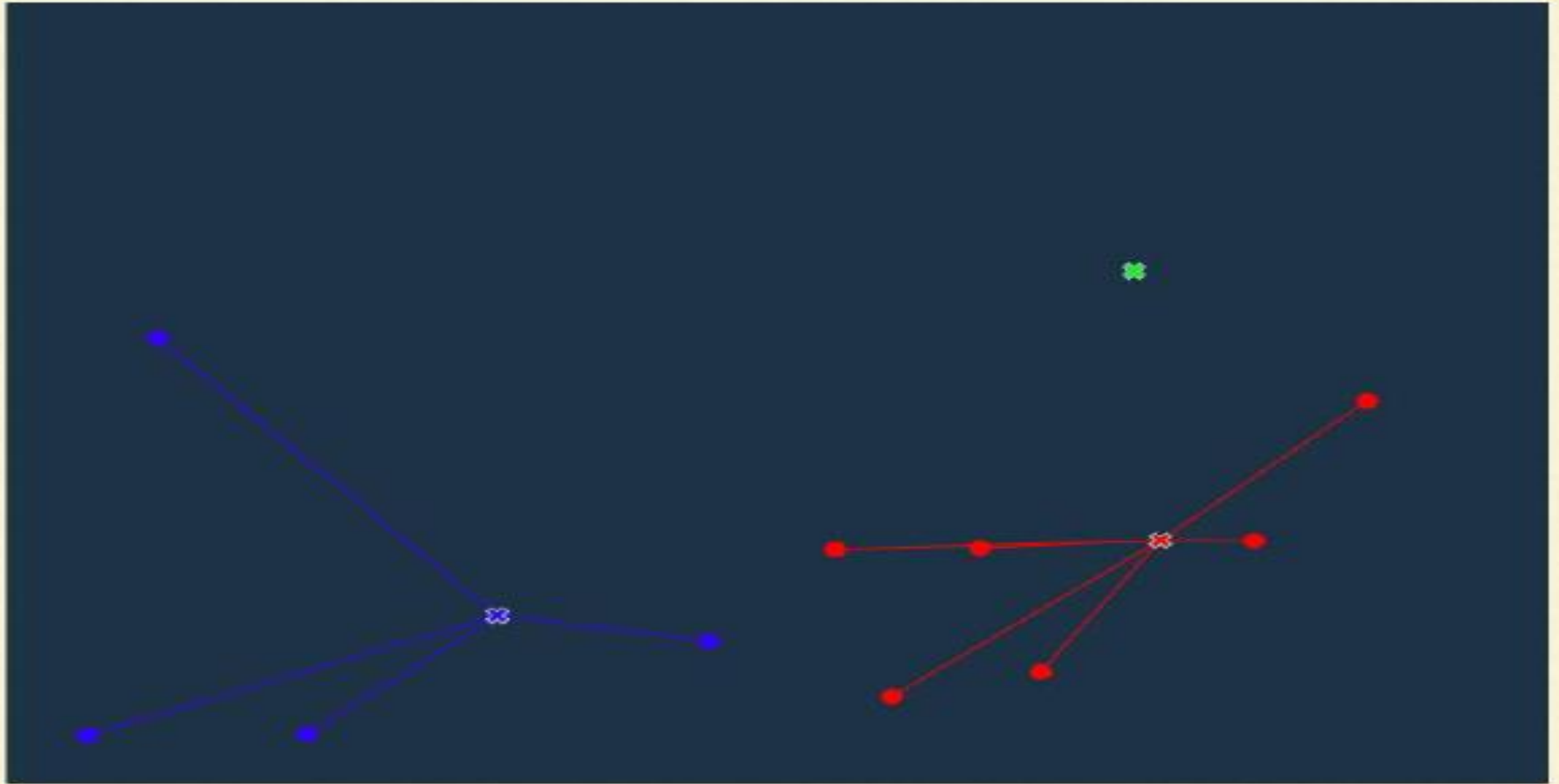


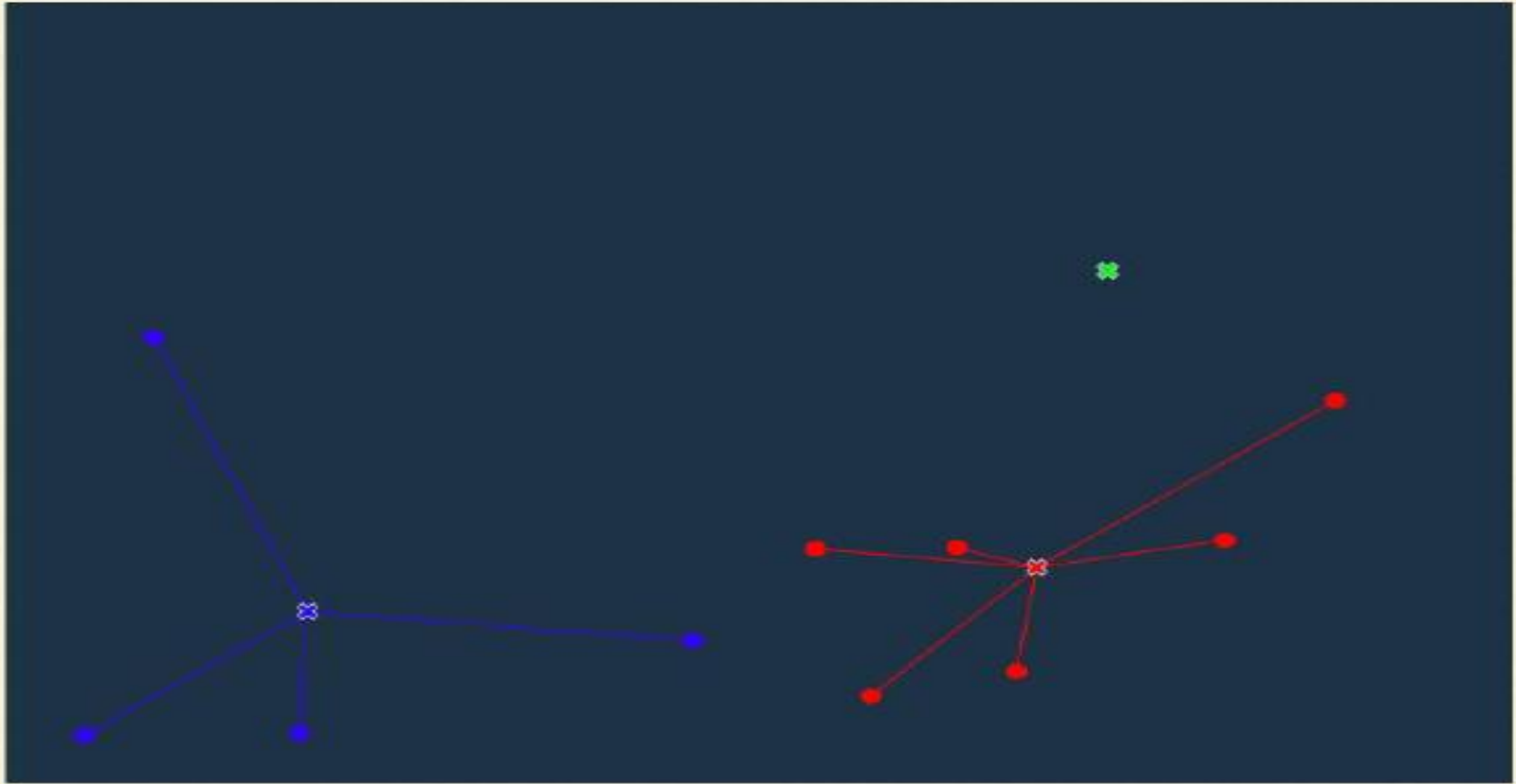


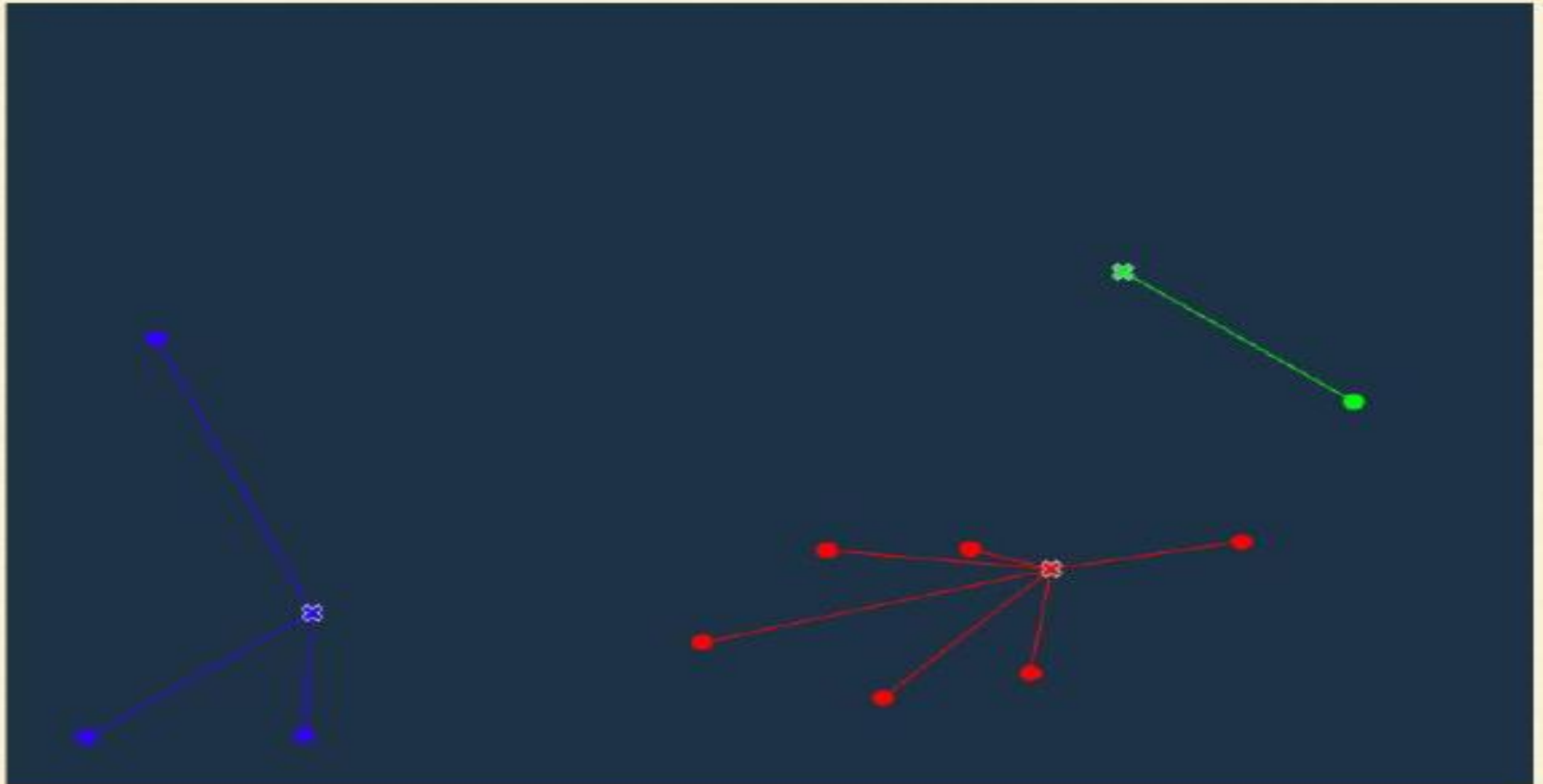


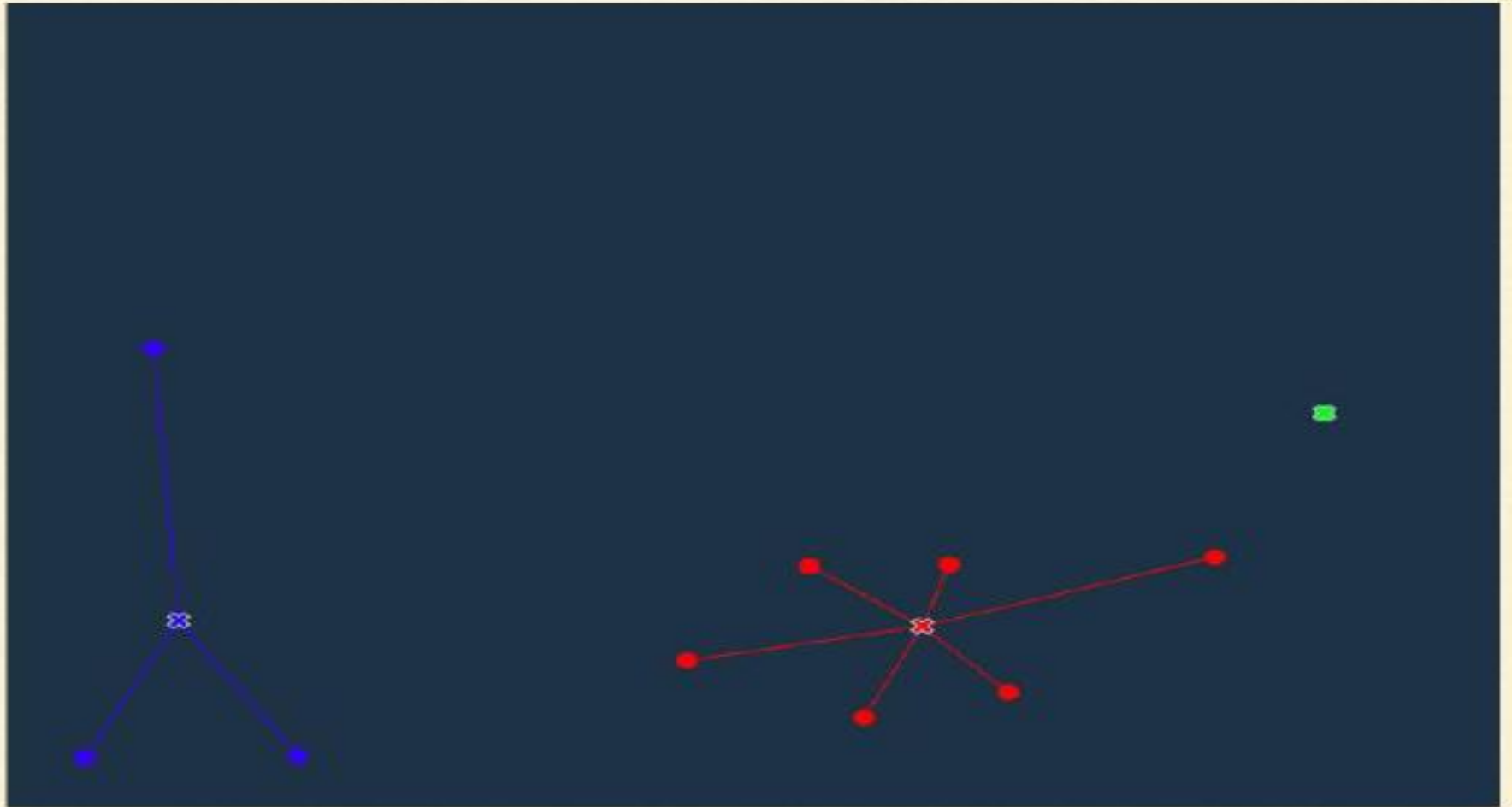


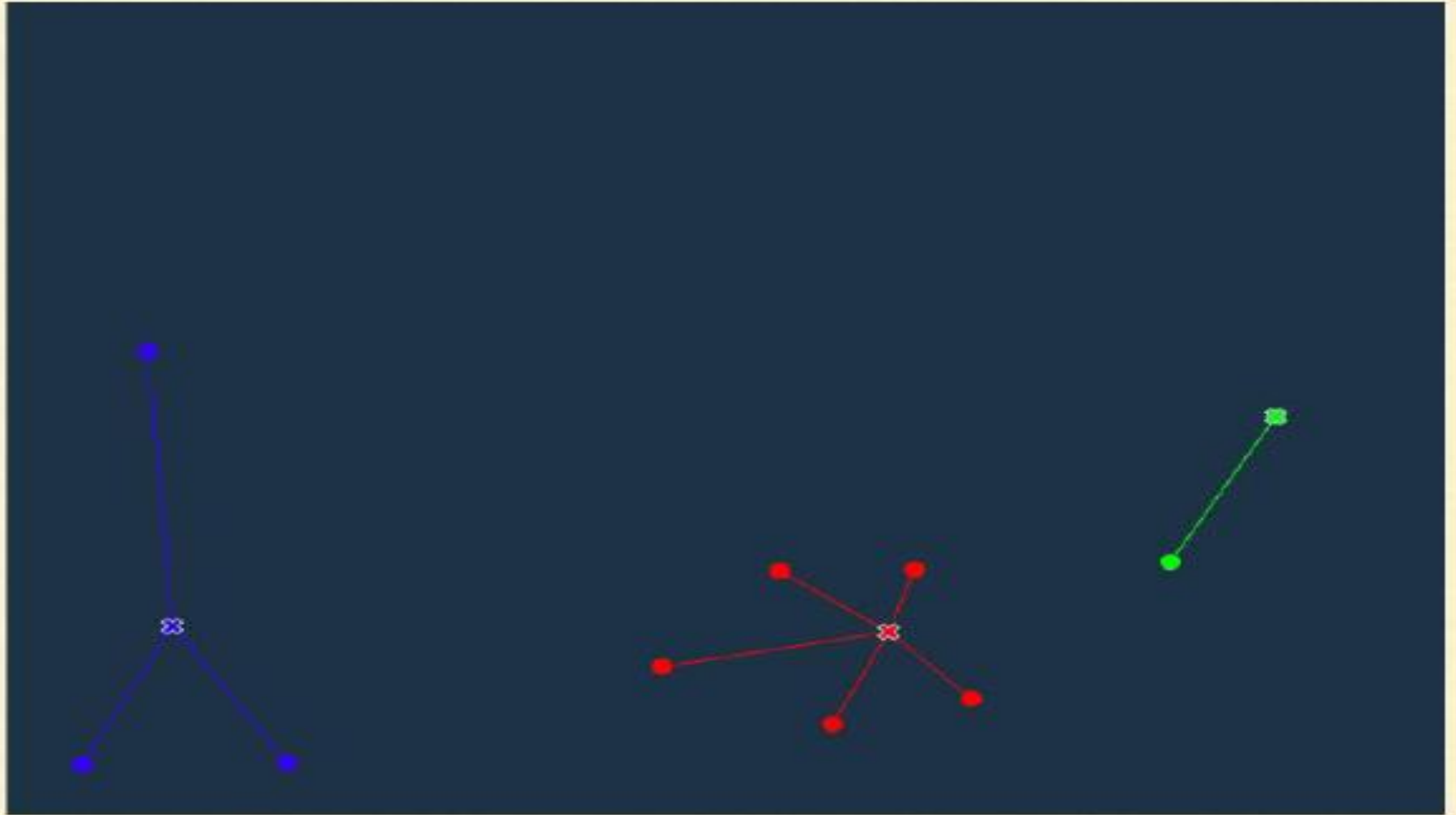


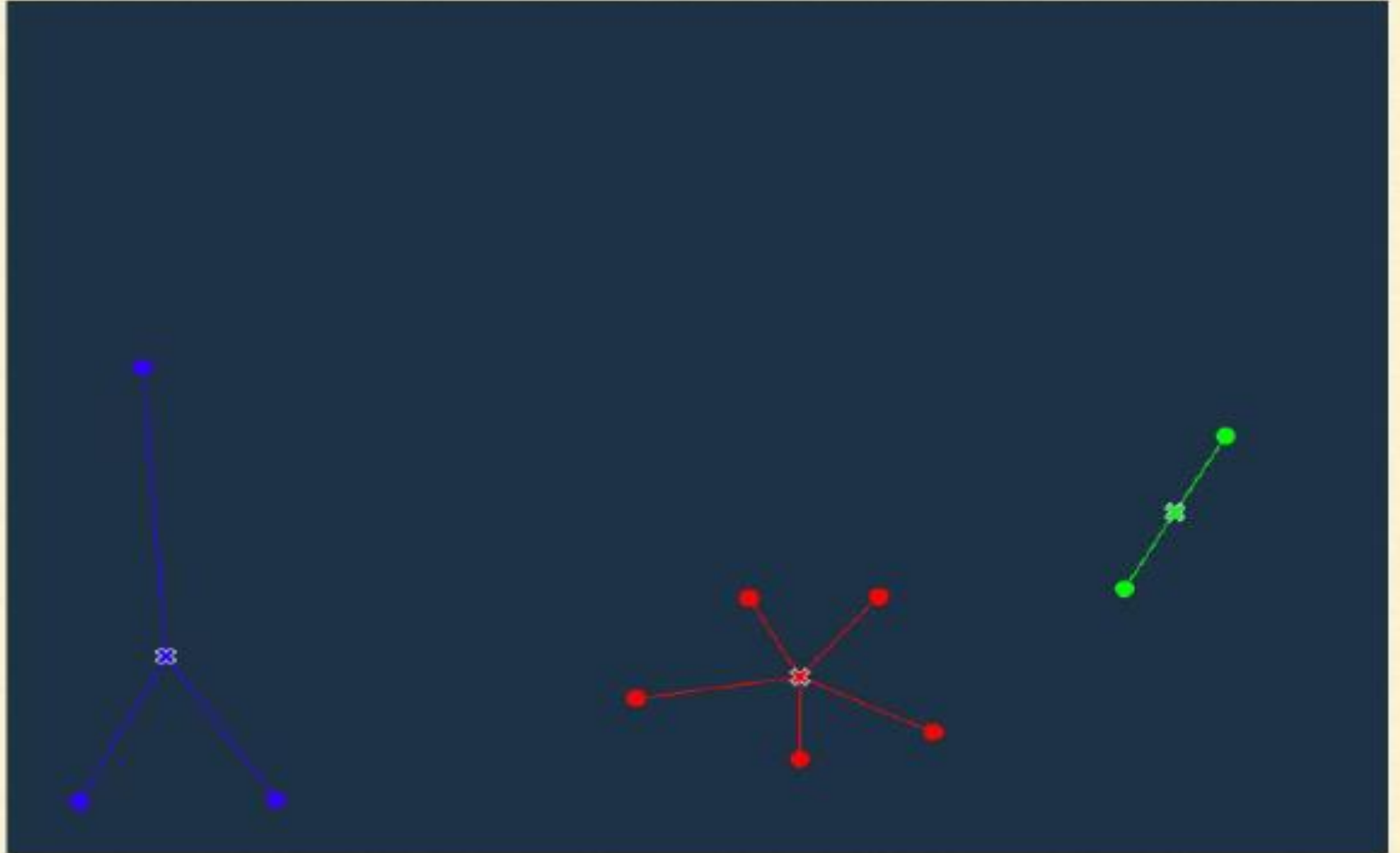












How to choose the right number of expected clusters (k)?

- Major difficulty found with k-means is the choice of the number of clusters.
- Different methods are proposed to solve this problem. Here, we provide a simple solution.
- The idea is to compute k-means clustering using different values of clusters k .
- Next, the within sum of squares is drawn according to the number of clusters.
- The location of a bend (knee) in the plot is generally considered as an indicator of the appropriate number of clusters.

-
- Two common methods for determining the optimal number of clusters in a dataset are the
 - “Elbow Method” and the
 - “Silhouette Score.”

Dr. Ritu Jain



-
- The Elbow Method and Silhouette Score determine the optimal number of clusters () in K-Means.
 - The Elbow Method plots Within-Cluster Sum of Squares (WCSS) against , selecting the "elbow" point where WCSS decreases slower.
 - The Silhouette Score measures how similar a point is to its own cluster versus others, with higher scores (closer to 1) indicating better-defined clusters



ELBOW METHOD

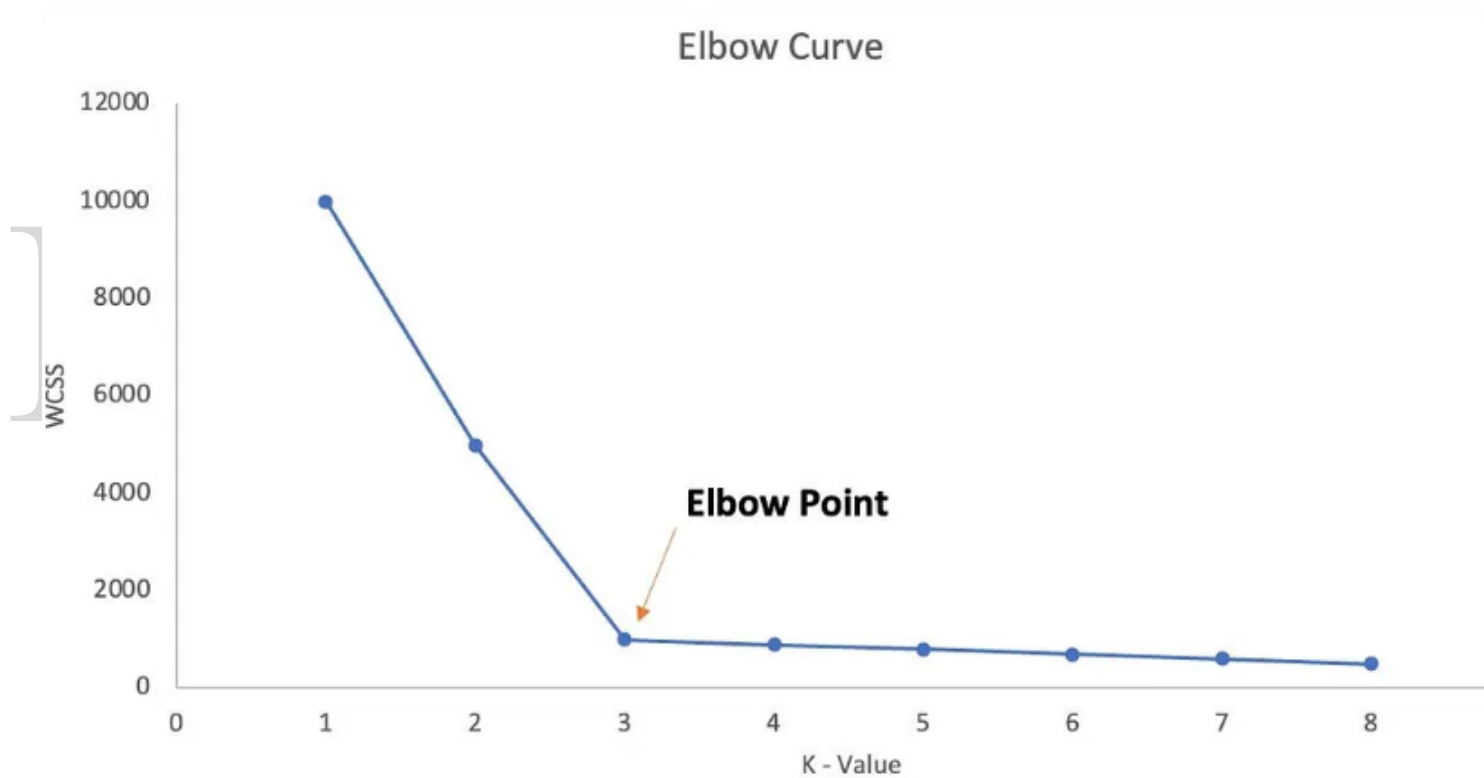
- The Elbow Method is a simple yet effective technique for finding the optimal number of clusters (k) in a dataset.
- *It relies on the intuition that as you increase the number of clusters, the within-cluster variation (also known as the Sum of Squared Distances or SSD) or WCSS (Within cluster sum squared) typically decreases.*
- *However, there is a point where adding more clusters does not significantly reduce the SSD.*
- This point is known as the “elbow point,” and it represents a trade-off between minimizing the WCSS and avoiding overfitting.



ELBOW METHOD

- The steps to apply the Elbow Method are as follows:
 1. Run the k-means clustering algorithm for a range of k values, typically from 1 to a reasonably large number.
 2. Calculate the SSD for each k.
 3. Plot the k values against their respective SSD values.
 4. Look for the “elbow” on the plot, which is the point where the SSD starts to level off.
- In the plot, the elbow point is where the rate of decrease in SSD slows down, forming an elbow-like bend. This bend indicates a suitable number of clusters for your dataset.



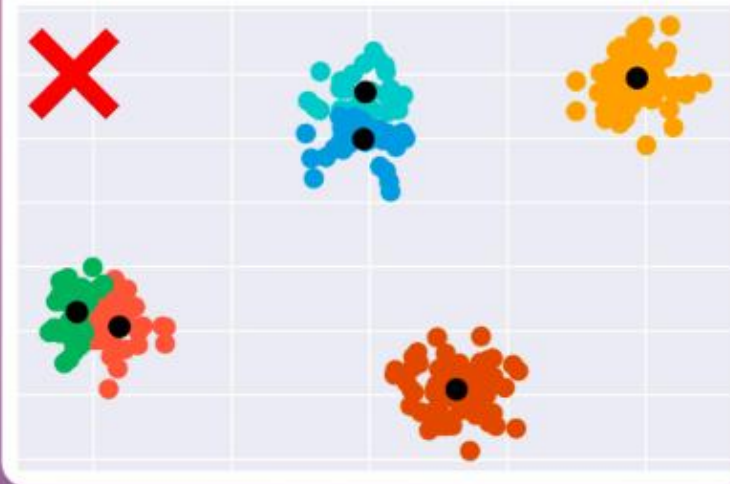


Silhouette Score

- The silhouette score is a metric used to evaluate the quality of clustering algorithms by measuring how similar an object is to its own cluster (cohesion) compared to other clusters (separation).
- **Ranging from -1 to +1 ,**
- Interpretation:
 - A score near +1 indicates, highly dense, well-separated clusters.
 - A score near 0 indicates point is near cluster boundaries or clusters overlap.
 - A negative score suggests points might have been assigned to the wrong cluster.

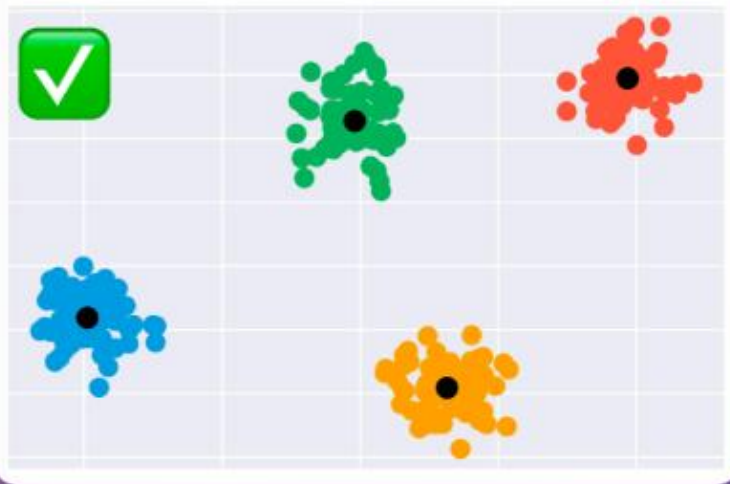


K-Means (Six Centroids)



Silhouette Score: 0.58

K-Means (Four Centroids)



Silhouette Score: 0.84

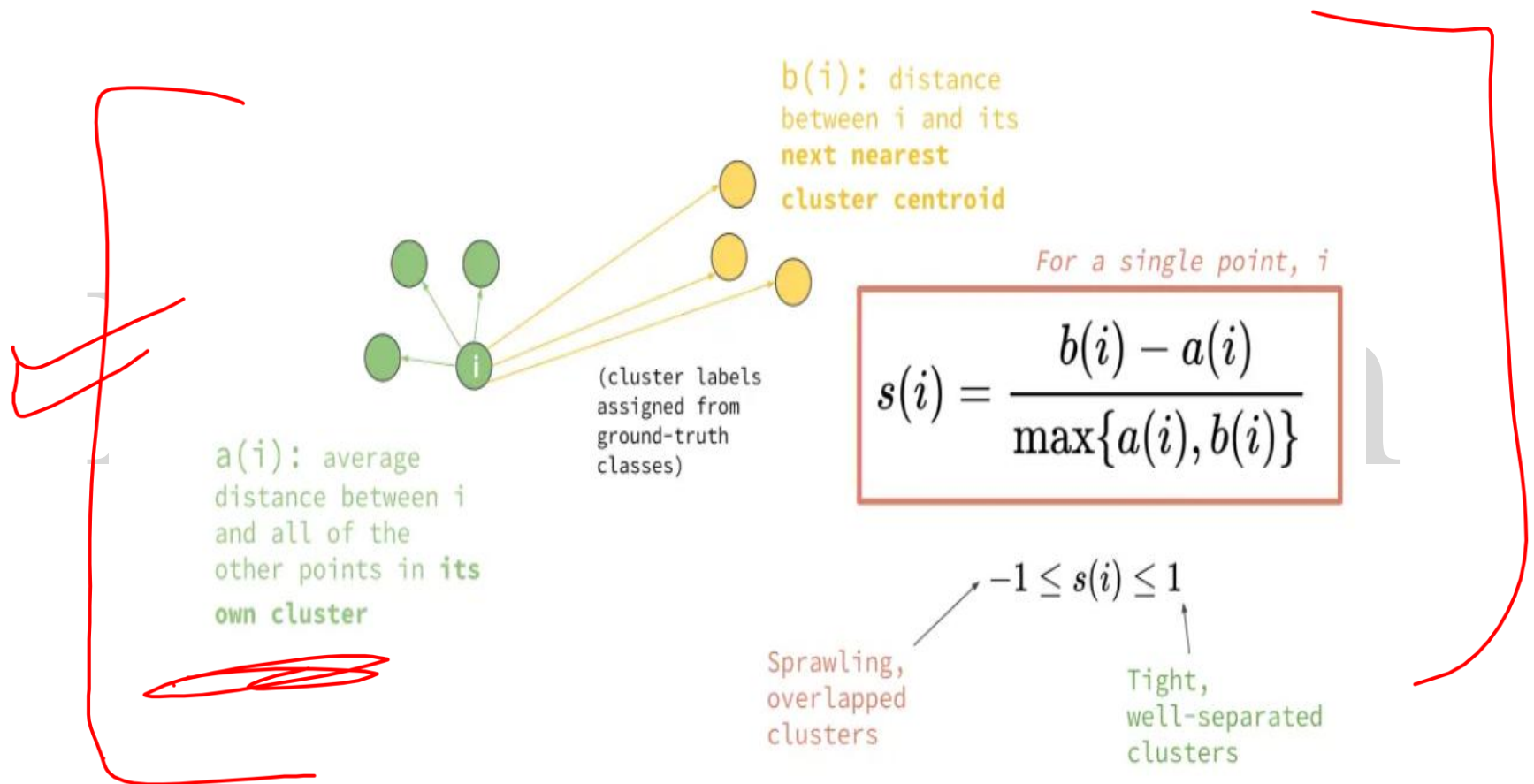
Higher score,
better clustering.

How the Silhouette Score Works

- The Silhouette Score measures how well each data point fits within its assigned cluster and how well-separated it is from other clusters. For each point, two key quantities are calculated:
- Intra-cluster distance ($a\{i\}$): This is the average distance between the data point and all other points in the same cluster. A smaller value indicates the point is closely aligned with its cluster.
- Nearest-cluster distance ($b\{i\}$): This is the average distance between the data point and all points in the nearest neighbouring cluster (the next best alternative). A larger value means the point is well-separated from other clusters.



Silhouette Score



Silhouette Score

Formula: For a single point i , the silhouette score $s(i)$ is:

$$s(i) = \frac{b(i) - a(i)}{\max\{a(i), b(i)\}}$$

- Where $a(i)$ is the mean distance between i and all other points in the same cluster, and $b(i)$ is the mean distance between i and all points in the nearest neighbor cluster.

- Usage: It is commonly used to determine the optimal number of clusters (k) by choosing the k that maximizes the average silhouette score.



Strengths and Limitations of Silhouette Score

- **Strengths**

- **Unsupervised Evaluation:** Works without the need for true cluster labels, making it ideal for unsupervised learning.
- **Promotes Good Clustering:** Rewards clusters that are both compact (cohesion) and well-separated (separation).
- **Visual Insight:** Silhouette plots provide a graphical understanding of cluster quality.

- **Limitations**

- **Distance Metric Sensitivity:** Works best with Euclidean distance; struggles with non-Euclidean metrics.
- **Cluster Shape Sensitivity:** Assumes spherical clusters; less effective for irregular shapes (e.g., in DBSCAN).
- **Scalability:** Computationally expensive for large datasets due to pairwise distance calculations.



Feature 	Elbow Method	Silhouette Method
Primary Metric	Within-Cluster Sum of Squares (WCSS) / Inertia.	Silhouette Coefficient.
Focus	Minimizes cluster cohesion (how tight points are to their centroid).	Balances cohesion and separation (how far clusters are from each other).
Interpretation	Look for the "elbow point" where the rate of decrease in WCSS sharply levels off.	Look for the highest average score . Scores range from -1 to +1.
Performance	Computationally faster; better for large datasets.	More computationally expensive; better for overlapping clusters.
Reliability	Often subjective; "elbows" can be ambiguous or non-existent in some data.	Generally considered more precise and objective.



Key Advantages of K-Means Clustering

- **Computational Efficiency:** K-Means is fast and efficient.
- **Simple to Implement:** The algorithm's logic is straightforward to understand and implement.
- **Scalability:** It scales well to large datasets and high-dimensional data.
- **Easy Interpretation:** It produces clear, interpretable results that are easy to visualize.
- **Guaranteed Convergence:** The algorithm is guaranteed to converge, meaning it will always find a solution.



Key Disadvantages of K-Means Clustering

- **Need to Choose k:** You must select the number of clusters (k) in advance, which can be difficult without prior knowledge.
- **Sensitivity to Initialization:** Different initial cluster centers (centroids) can result in different final clusters (local optima).
- **Outlier Sensitivity:** Outliers can significantly distort the resulting centroids, leading to poorer clustering results.
- **Assumption of Spherical Clusters:** K-Means assumes that clusters are spherical and roughly the same size, struggling with complex, non-linear, or elongated shapes.
- **Varying Densities:** It does not perform well on data with varying cluster densities.
- **Scaling Dependence:** Data normalization or standardization is necessary, as the algorithm relies on distance metrics.



How initialization of centroids affects K-Means performance

- Centroid initialization significantly dictates the quality and speed of K-Means clustering because the algorithm is a hill-climbing optimization that converges to a local minimum rather than a guaranteed global optimum.
- Core Effects of Initialization
 - Convergence to Local Optima: Poor initial placement often leads to "suboptimal clusters" where the algorithm gets stuck in a local minimum, failing to find the most accurate natural groupings.
 - Computational Efficiency: Strategic initialization (like K-Means++) reduces the total number of iterations required to reach stability, leading to faster execution times.
 - Consistency and Stability: Random initialization causes the final cluster boundaries to vary across different runs of the same data. This lack of reproducibility can be mitigated by multiple initializations (`n_init`) and selecting the one with the lowest inertia.
 - Cluster Quality (WCSS): Effective initialization minimizes the Within-Cluster Sum of Squares (WCSS), ensuring data points are closer to their respective centroids.

Common Initialization Strategies of centroids in K-Means clustering

- **Common Initialization Strategies**
 - **Random Initialization (Forgy Method):** Randomly selects data points as the initial centers. While simple, it often results in centroids falling too close together or in empty regions, leading to poor results.
 - **K-Means++:** A smart probabilistic method that selects the first centroid randomly and then chooses subsequent centroids far from those already selected. This ensures a better spread across the data space, improving both accuracy and speed.
 - **Deterministic Approaches:** Methods like selecting the most distant points or using density-aware logic aim to place centroids in high-density areas to avoid outliers



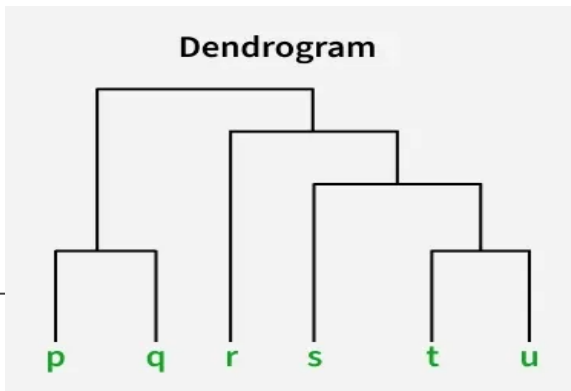
Hierarchical Clustering

- Hierarchical clustering is an unsupervised learning algorithm used to group similar data points into clusters.
- It builds a multilevel hierarchy of clusters by either merging smaller clusters into larger ones (**agglomerative**) or dividing a large cluster into smaller ones (**divisive**).
- It builds a tree-like structure (**dendrogram**) that helps visualize relationships and decide the optimal number of clusters.
- Does not require pre-selecting the number of clusters.
- Commonly applied in data exploration and pattern discovery.
- It is commonly used in pattern recognition, customer segmentation and image grouping.



Dendrogram

- A dendrogram is a visual representation that illustrates the arrangement of clusters and their relationships with each other.
- A dendrogram is a tree-like structure that visualizes the process of hierarchical clustering.
- Each level of the tree represents a merge or split operation, and the height of the branches represents the distance (or dissimilarity) at which clusters were joined.
- It shows how individual data points or groups of data merge together.
- The bottom shows each data point as its own group and as we move up, similar groups are combined.
- The lower the merge point, the more similar the groups are. It helps us see how things are grouped step by step.
- *Lower heights indicate clusters joined at smaller distances, thus representing higher similarity.*



Importance of Dendrogram

- **Importance of Dendrogram:**

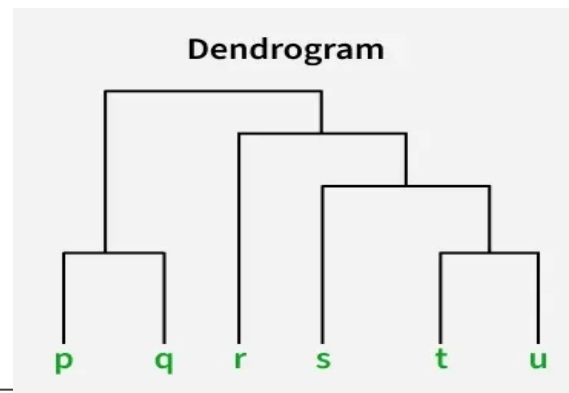
- Allows visual inspection of clusters at different levels.
- By "cutting" the dendrogram at a certain height, you can choose the number of clusters that best fits the data.

Dr. Ritu Jain



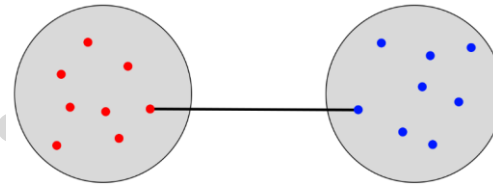
Dendrogram

- At the bottom of the dendrogram the points P, Q, R, S and T are all separate.
- As we move up, the closest points are merged into a single group.
- The lines connecting the points show how they are progressively merged based on similarity.
- The height at which they are connected shows how similar the points are to each other; the shorter the line the more similar they are

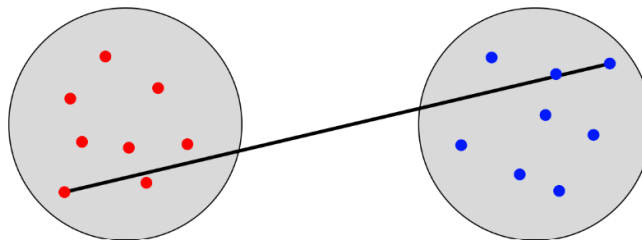


Linkage criteria

- Linkage criteria determine how distances between clusters are calculated during the merging process.
 - Single linkage (minimum):** This criterion looks at the distance between the closest points of two clusters. This can result in elongated, chain-like clusters.

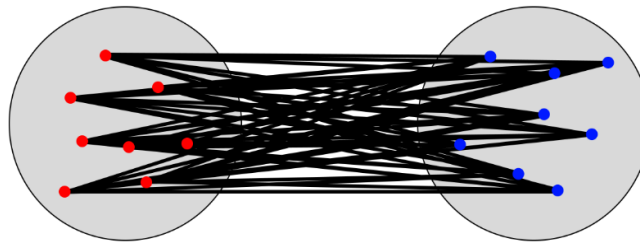


- Complete linkage (maximum):** This criterion looks at the distance between the farthest points of two clusters. This tends to produce compact, evenly sized clusters.



Linkage criteria

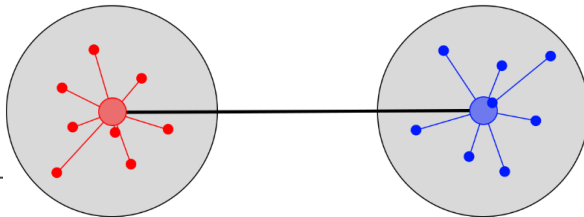
- ✓ **Average linkage (mean):** This criterion looks at the average distance between all points in one cluster to all points in another. This gives a balance between single and complete linkage.



- **Ward's method:** Ward's method is a variation of the average linkage approach that minimizes the sum of squared differences within all clusters. It tends to produce more evenly sized clusters compared to other methods. It also often results in well-separated, spherical clusters.

Linkage criteria: Ward Method

- Ward's method involves the formation of clusters in a way that minimizes the variance within each cluster.
- At each step during the clustering process, two clusters are merged in a manner that results in the smallest increase in the total sum of squared distances between the points of the resulting cluster.
- Of the different linkage methods provided, this one is often the most computationally intensive, but can perform well when data contain clusters of varying sizes and densities.
- Note however that Ward's method can be sensitive to outliers as these will have large impacts on the total sum of squared distances.
- In essence, Ward's method attempts to minimize the red/blue lines within clusters in the image below, and as a result maximizes the black lines between clusters.



- **Single Linkage**

$$D(c_1, c_2) = \min D(x_1, x_2)$$

Minimum distance or distance between closest elements in clusters



- **Complete Linkage**

$$D(c_1, c_2) = \max D(x_1, x_2)$$

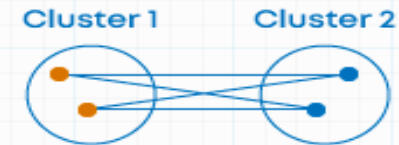
Maximum distance between elements in clusters



- **Average Linkage**

$$D(c_1, c_2) = \frac{1}{|c_1|} \frac{1}{|c_2|} \sum \sum D(x_1, x_2)$$

Average of the distances of all pairs



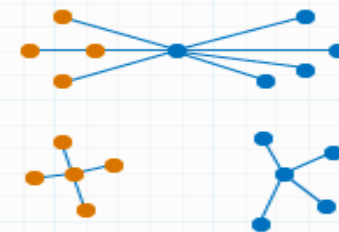
- **Centroid Method**

Combining clusters with minimum distance between the centroids of the two clusters



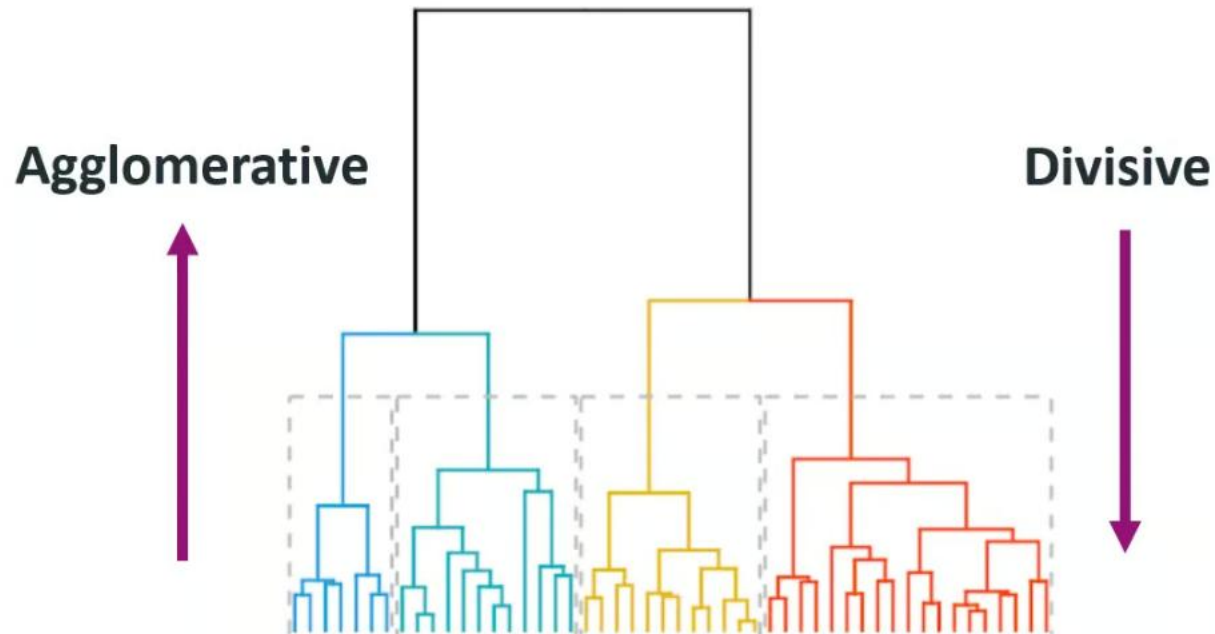
- **Ward's Method**

- Combining clusters where increase in within cluster variance is to the smallest degree.
- Objective is to minimize the total within cluster variance



Types of Hierarchical Clustering

- **Agglomerative (Bottom-Up)**
- **Divisive (Top-Down)**



in

Hierarchical vs K-Means clustering

When to Use K-Means Clustering

- **Large Datasets:** When efficiency and speed are required, as it has a lower computational complexity.
- **Spherical Clusters:** When you suspect the clusters are roughly spherical or convex in shape.
- **Known Number of Clusters:** When you know (k) or can estimate the number of clusters beforehand.
- **Customer Segmentation:** Efficiently grouping millions of customers based on spending habits or demographics.
- **Image Segmentation:** Dividing an image into segments based on pixel intensity.



Hierarchical vs K-Means clustering

When to Use Hierarchical Clustering

- **Unknown Number of Clusters:** When you do not know the number of clusters and need to determine the optimal number using a dendrogram.
- **Small to Medium Datasets:** Because it is computationally expensive and slow for very large datasets.
- **Hierarchical Structure:** When data naturally forms a hierarchy (e.g., taxonomy, taxonomy of products, hierarchical organization of employees).
- **Visual hierarchy:** Good for small datasets or when a visual hierarchy is needed.
- **Bioinformatics:** Creating phylogenetic trees or analyzing gene expression patterns, where understanding the relationship between clusters is crucial.
- **Small-scale Data Analysis:** When detailed, exploratory analysis of relationships is more important than speed



K-Means vs Hierarchical clustering

Feature	<u>K-Means</u>	<u>Hierarchical Clustering</u>
✓ Number of Clusters (k)	Must be defined in advance.	Determined later via dendrogram.
✓ Data Size	Good for large datasets.	Good for small datasets.
✓ Structure	Flat (non-nested).	Tree-like (nested).
✓ Performance	Faster, faster convergence.	Slower, computationally expensive.
✓ Cluster Shape	Works best for spherical shapes.	Can identify complex, non-spherical shapes.

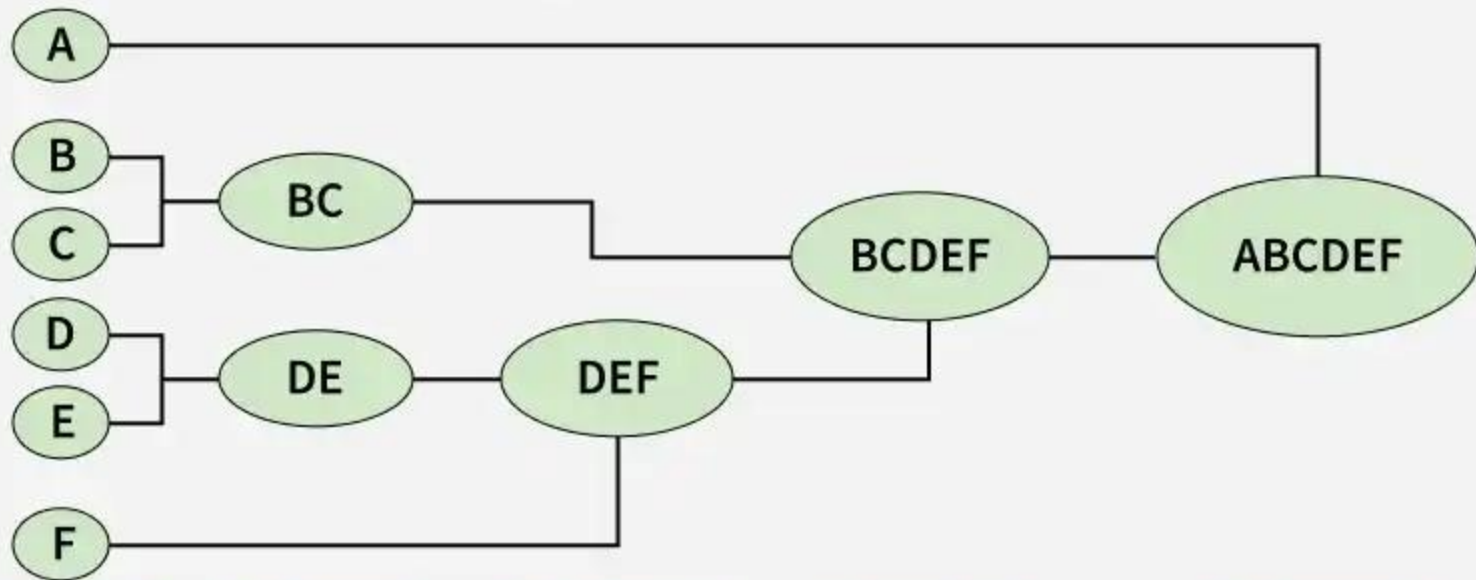


Agglomerative (Bottom-Up)

- Agglomerative clustering starts with each data point as an individual cluster and iteratively merges the closest pair of clusters until only one cluster remains or until a stopping condition is met (like a desired number of clusters).
- This method is also called bottom-up because it starts from the bottom (individual data points) and builds up to the top (final cluster).
- This method is widely used due to its simplicity and ease of implementation.



Bottom-Up (Agglomerative Clustering)



Step 1

Step 2

Step 3

Step 4

Step 5



Workflow for Hierarchical Agglomerative clustering

- **Start with individual points:** Each data point is its own cluster. For example if we have 5 data points we start with 5 clusters each containing just one data point.
- **Calculate distances between clusters:** Calculate the distance between every pair of clusters. Initially since each cluster has one point this is the distance between the two data points.
- **Merge the closest clusters:** Identify the two clusters with the smallest distance and merge them into a single cluster.
- **Update distance matrix:** After merging we now have one less cluster. Recalculate the distances between the new cluster and the remaining clusters.
- **Repeat steps 3 and 4:** Keep merging the closest clusters and updating the distance matrix until we have only one cluster left.
- **Create a dendrogram:** As the process continues we can visualize the merging of clusters using a tree-like diagram called a dendrogram. It shows the hierarchy of how clusters are merged.



Calculate distances between clusters in agglomerative clustering

- Calculating distances between clusters in agglomerative clustering involves using a linkage method—such as Single, Complete, Average, or Ward—to determine the proximity between sets of points.
- Initially, pairwise distances (e.g., Euclidean or Manhattan) are calculated, and the two closest clusters are merged iteratively until all points are grouped, updating the distance matrix at each step



Divisive (Top-Down)

- Divisive clustering starts with all data points in a single cluster and recursively splits them into smaller clusters. The process continues until each data point is in its own individual cluster.
- It is also known as a top-down approach, as it starts at the top (single cluster) and breaks it down into smaller clusters.
- **Step-by-step guide:**
 1. Start with one large cluster containing all data points.
 2. Split the cluster into two that are as different as possible.
 3. Reapply the splitting process recursively to each resulting cluster.
- Divisive methods are typically more computationally expensive due to their recursive nature, and accuracy depends greatly on the splitting strategy. Agglomerative methods are more common due to ease of implementation and widespread software support.

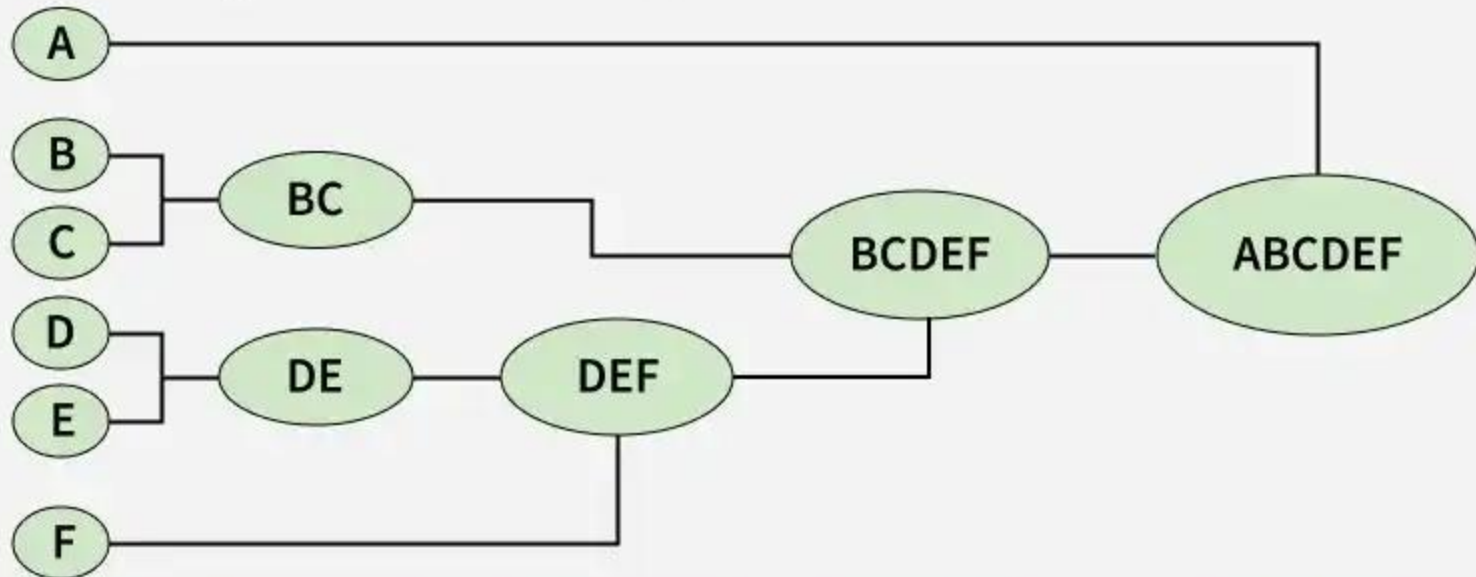


Workflow for Hierarchical Divisive clustering

- **Start with all data points in one cluster:** Treat the entire dataset as a single large cluster.
- **Split the cluster:** Divide the cluster into two smaller clusters. The division is typically done by finding the two most dissimilar points in the cluster and using them to separate the data into two parts.
- **Repeat the process:** For each of the new clusters, repeat the splitting process: Choose the cluster with the most dissimilar points and split it again into two smaller clusters.
- **Stop when each data point is in its own cluster:** Continue this process until every data point is its own cluster or the stopping condition (such as a predefined number of clusters) is met.



Top-Down (Divisive Clustering)



Step 5

Step 4

Step 3

Step 2

Step 1



Key Usage of Hierarchical Clustering

- **Customer Segmentation:** Grouping customers based on purchasing behavior, demographics, or browsing patterns.
- **Bioinformatics:** Organizing genes with similar expression patterns or classifying species, often used with heatmaps for genetic analysis.
- **Image Processing/Grouping:** Identifying similar items or grouping images in computer vision applications.
- **Document Clustering:** Organizing large collections of documents into topic hierarchies.
- **Social Network Analysis:** Mapping relationships and community structures within social networks.
- **Data Exploration:** Creating visual structures (dendrograms) to understand hierarchical relationships in small-to-medium-sized datasets.



Imp Note: Hierarchical Clustering

- **AGNES (Agglomerative Nesting)** is a bottom-up hierarchical clustering algorithm in machine learning that treats each data point as a singleton cluster and merges the closest pairs of clusters recursively until all points unite into one root cluster.
- **DIANA (Divisive Analysis)** is a top-down hierarchical clustering algorithm in machine learning that starts with all data points in one cluster and recursively splits them until every object is in its own cluster.



Agglomerative Hierarchical Clustering using Complete Linkage Numerical

- Given a one-dimensional data set $\{1, 5, 8, 10, 2\}$, use the agglomerative clustering algorithms with the complete link with Euclidean distance to establish a hierarchical grouping relationship.

By using the cutting threshold of 5, how many clusters are there?



Agglomerative Hierarchical Clustering using Complete Linkage Numerical

- $$\text{Euclidean distance} = \sqrt{(x_2 - x_1)^2}$$

- In order to use the agglomerative algorithm,
- we need to calculate the distance matrix.
- One-dimensional data set {1, 5, 8, 10, 2}

	1	5	8	10	2
1	0	4	7	9	1
5	4	0	3	5	3
8	7	3	0	2	6
10	9	5	2	0	8
2	1	3	6	8	0

- Minimum distance is between point 1 and 5.
- So, merge them into one cluster {1,5}

	1	2	3	4	5
1	0	4	7	9	1
2	4	0	3	5	3
3	7	3	0	2	6
4	9	5	2	0	8
5	1	3	6	8	0



- Now find the distance between cluster {1,5} and point 2 using formulae of complete linkage.

$$\rightarrow d(2, \{1,5\}) = \max\{d(2,1), d(2,5)\} = \max\{4, 3\} = 4$$

- And do it for point 3 and 4 also

$$\rightarrow d(3, \{1,5\}) = \max\{d(3,1), d(3,5)\} = \max\{7, 6\} = 7$$

$$\rightarrow d(4, \{1,5\}) = \max\{d(4,1), d(4,5)\} = \max\{9, 8\} = 9$$

- Now, check the distance between point 3 and point 4 is minimum.

Hence, they merge together to form a cluster {3, 4}.

Using the complete link, we have the distance between different points/clusters as follows:

$$\rightarrow d(\{1,5\}, \{3, 4\}) = \max\{d(\{1,5\}, 3), d(\{1,5\}, 4)\} = \max\{7, 9\} = 9$$

$$\rightarrow d(2, \{3,4\}) = \max\{d(2,3), d(2,4)\} = \max\{3, 5\} = 5$$

Thus, we can update the distance matrix, where row 2

corresponds to point 2, rows 1 and 3 correspond to clusters {1, 5}

85 and {3, 4}, as follows:

	1,5	2	3	4
1,5	0	4	7	9
2	4	0	3	5
3	7	3	0	2
4	9	5	2	0

	1,5	2	3,4
1,5	0	4	9
2	4	0	5
3,4	9	5	0

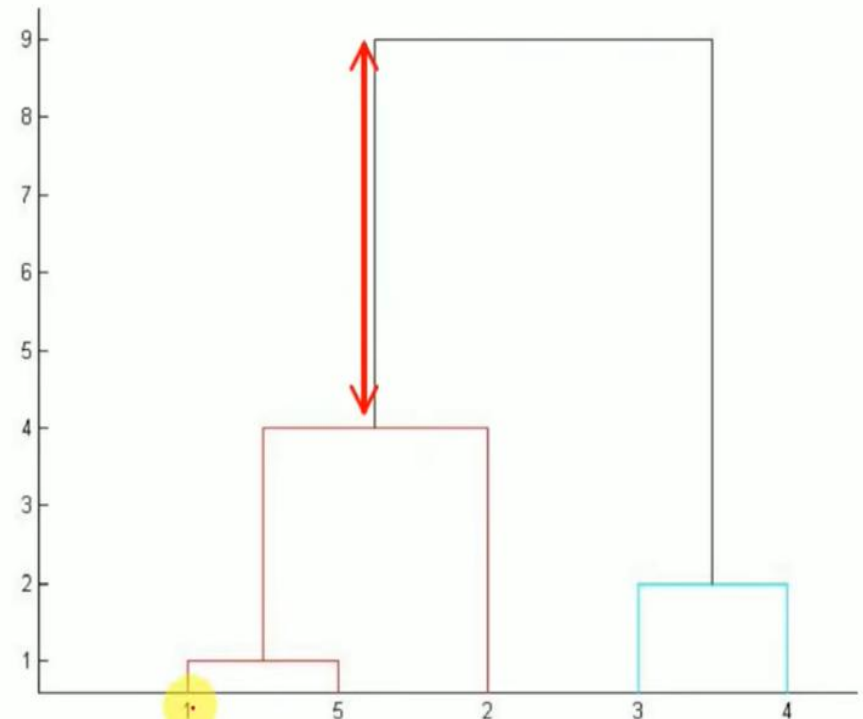


Following the same procedure, we merge point 2 with the cluster {1, 5} to form {1, 2, 5} and update the distance matrix as follows:

D:

$$\begin{matrix} & [1,5],2 & [3,4] \\ [1,5],2 & \begin{bmatrix} 0 & 9 \end{bmatrix} \\ [3,4] & \begin{bmatrix} 9 & 0 \end{bmatrix} \end{matrix}$$

- After increasing the distance threshold to 9, all clusters would merge.



DBSCAN Clustering

- **DBSCAN (Density-Based Spatial Clustering of Applications with Noise)**
- The main concept of DBSCAN algorithm is to locate regions of high density that are separated from one another by regions of low density.
- It identifies clusters as dense regions in the data space separated by areas of lower density.
- It groups data points that are closely packed together and marks outliers as noise based on their density in the feature space.
- Unlike K-Means or hierarchical clustering which assumes clusters are compact and spherical, DBSCAN perform well in handling real-world data irregularities such as:
 - **Arbitrary-Shaped Clusters:** Clusters can take any shape not just circular or convex.
 - **Noise and Outliers:** It effectively identifies and handles noise points without assigning them to any cluster.



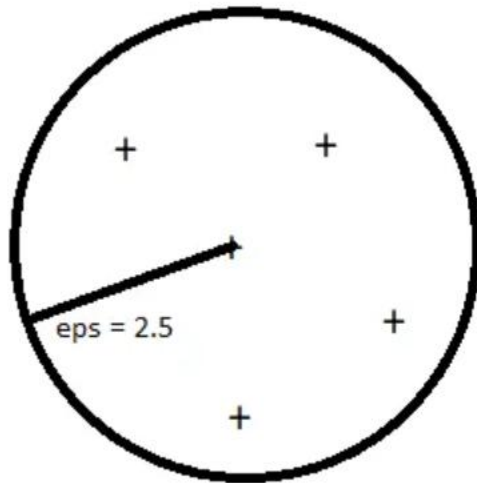
DBSCAN Clustering

- There are two parameters in DBSCAN:
 - minPoints and
 - eps
- eps: *specifies how close points should be to each other to be considered a part of a cluster. It means that if the distance between two points is lower or equal to this value (eps), these points are considered to be neighbors.*
- minPoints: *the minimum number of data points to form a dense region/ cluster. For example, if we set the minPoints parameter as 5, then we need at least 5 points to form a dense region.*



DBSCAN Clustering

- S

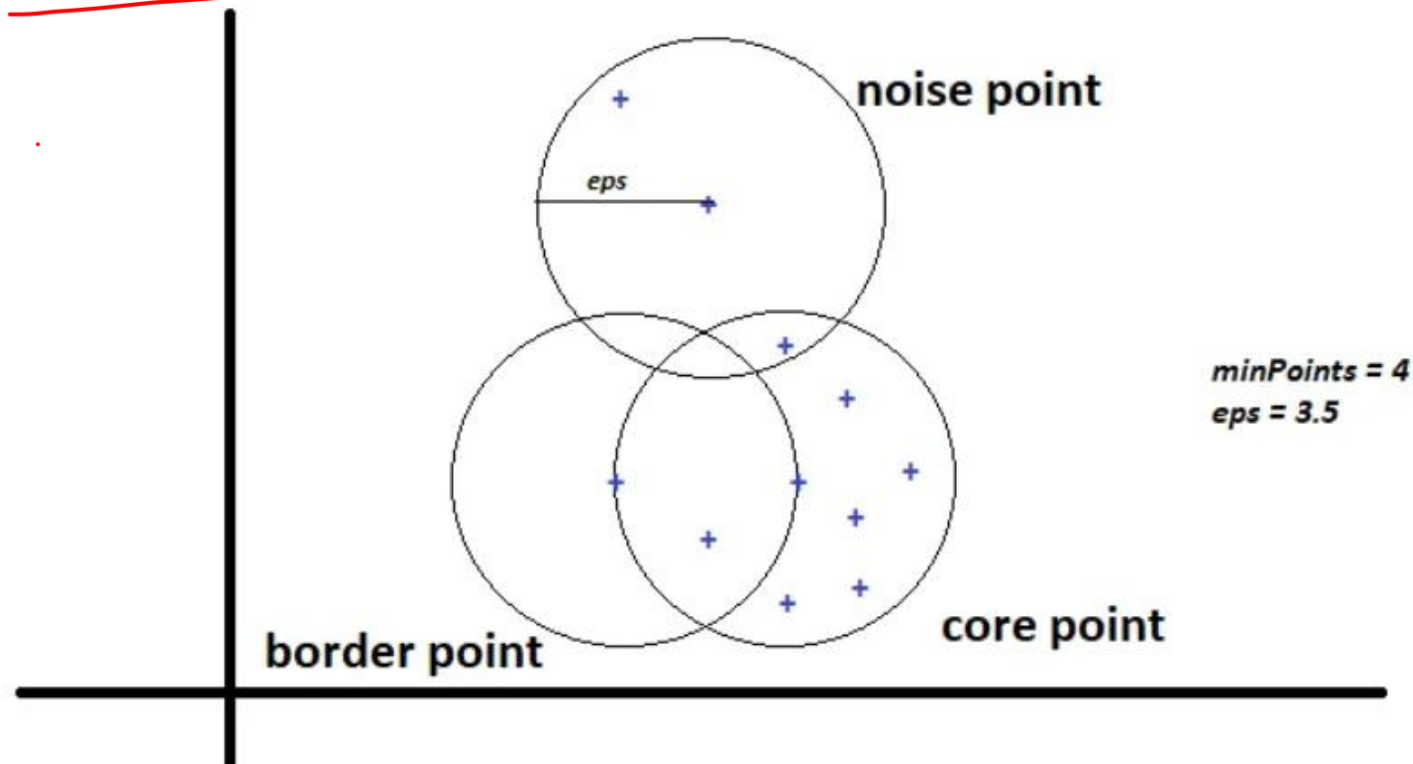


Let the minPoints = 3, and eps = 2.5
So, Density = 5
i.e density > minPoints

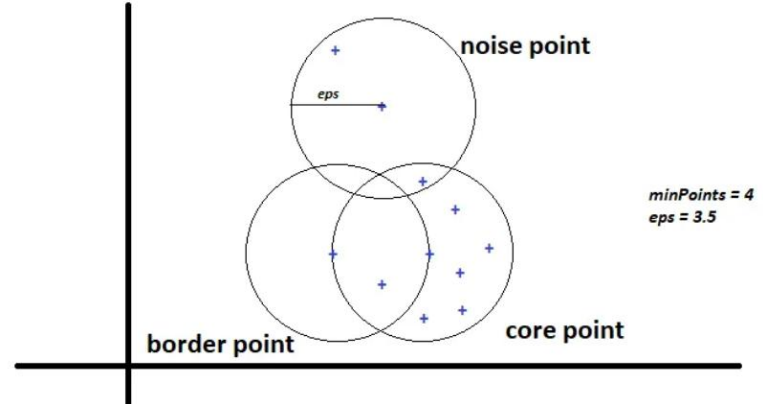
Hence, it's a "Dense Region"

DBSCAN Clustering

- Based on the two parameters, the points are classified as ***Core point***, ***Border point*** and ***Noise point***.



DBSCAN Clustering: Core Point, Border Point



- **Core Point**

- A point is a core point if it has more than a specified number of minPoints within eps radius around it or $|N(p)| \geq \text{minPoints}$.
- Core Point always belongs in a dense region.
- For example, let's consider 'p' is set to be a core point if 'p' has $\geq \text{minPoints}$ in an eps radius around it.

- **Border Point**

- A point is a border point if it has fewer than minPoints within eps, but is in the neighborhood of a *core point*.
- For example, p is set to be a border point if 'p' is not a core point. i.e 'p' has $< \text{minPoints}$ in eps radius. But 'p' should belong to the neighborhood 'q'. Where 'q' is a core point.
- $p \in \text{neighborhood of } q$ and $\text{distance}(p, q) \leq \text{eps}$.

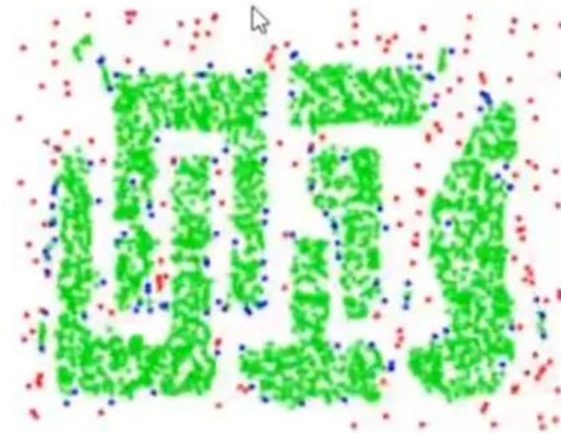
- **Noise Point**

- A noise point is any point that is not a core point or a border point.

I



Original Points



Point types: **core**,
border and **noise**

Eps = 10, MinPts = 4

1



Density Edge and the Density Connected Points:

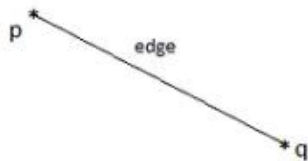
- **Density Edge:**

If p and q both are core points and distance between $(p, q) \leq \text{eps}$ then we can connect p, q vertex in a graph and call it “***Density Edge***”.

- **Density Connected Points:**

Two points p and q are said to be density connected points if both p and q are core points and they exist a path formed by density edges connecting point (p) to point(q).

Density edge



- i) p and q both are core points
- ii) $\text{distance}(p, q) \leq \text{eps}$

Density Connected Points



- i) p and q both are core points
- ii) $\text{distance}(p', p'') \leq \text{eps}$

DBSCAN Algorithm: Step-by-Step Algorithm Execution

1. **Select a Random Point:** The algorithm picks an arbitrary unvisited point from the dataset.
2. **Check Neighborhood Density:** It identifies all points within the eps distance of the selected point.
3. **Classify the Point:**
 1. **Core Point:** If the number of points in the neighborhood (including itself) is $\geq \text{minpts}$, a new cluster is started.
 2. **Noise Point:** If there are fewer than minpts neighbors, the point is temporarily labeled as noise. Note: A noise point can later become a border point if it is within range of a different core point.



DBSCAN Algorithm: Step-by-Step Algorithm Execution (contd)

4. **Expand the Cluster:**

1. If the starting point is a Core Point, all its neighbors are added to the cluster.
2. The algorithm then checks the neighborhood of every newly added point. If any of these "neighbor points" are also core points, their own neighborhoods are also merged into the cluster.
3. **Border Points** (points that have fewer than neighbors but are within reach of a core point) are added to the cluster but do not expand it further.

5. **Repeat:** Once the cluster cannot be expanded further, the algorithm selects the next unvisited point from the dataset and repeats the process.

6. **Finalize Outliers:** After all points have been visited, any points not assigned to a cluster are finalized as Noise.

Visualization of DBSCAN Clustering:

<https://www.naftaliharris.com/blog/visualizing-dbscan-clustering/>



D Y PATIL
UNIVERSITY
NAVI MUMBAI

- **Simplified DBSCAN Algorithm**

- **Step 1** — Identify all points as either core point, border point or noise point.

- **Step 2** — For all of the unclustered core points.

- **Step 2a** — Create a new cluster.

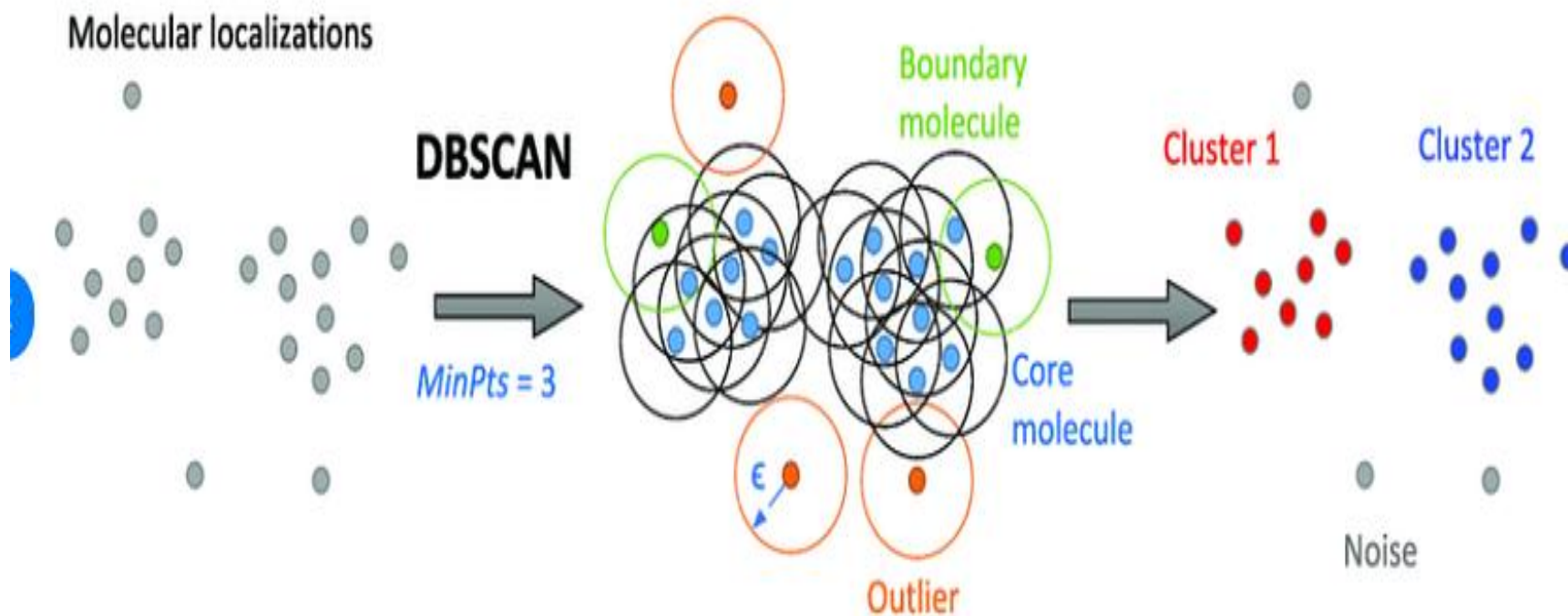
- **Step 2b** — add all the points that are unclustered and density connected to the current point into this cluster.

- **Step 3** — For each unclustered border point assign it to the cluster of nearest core point.

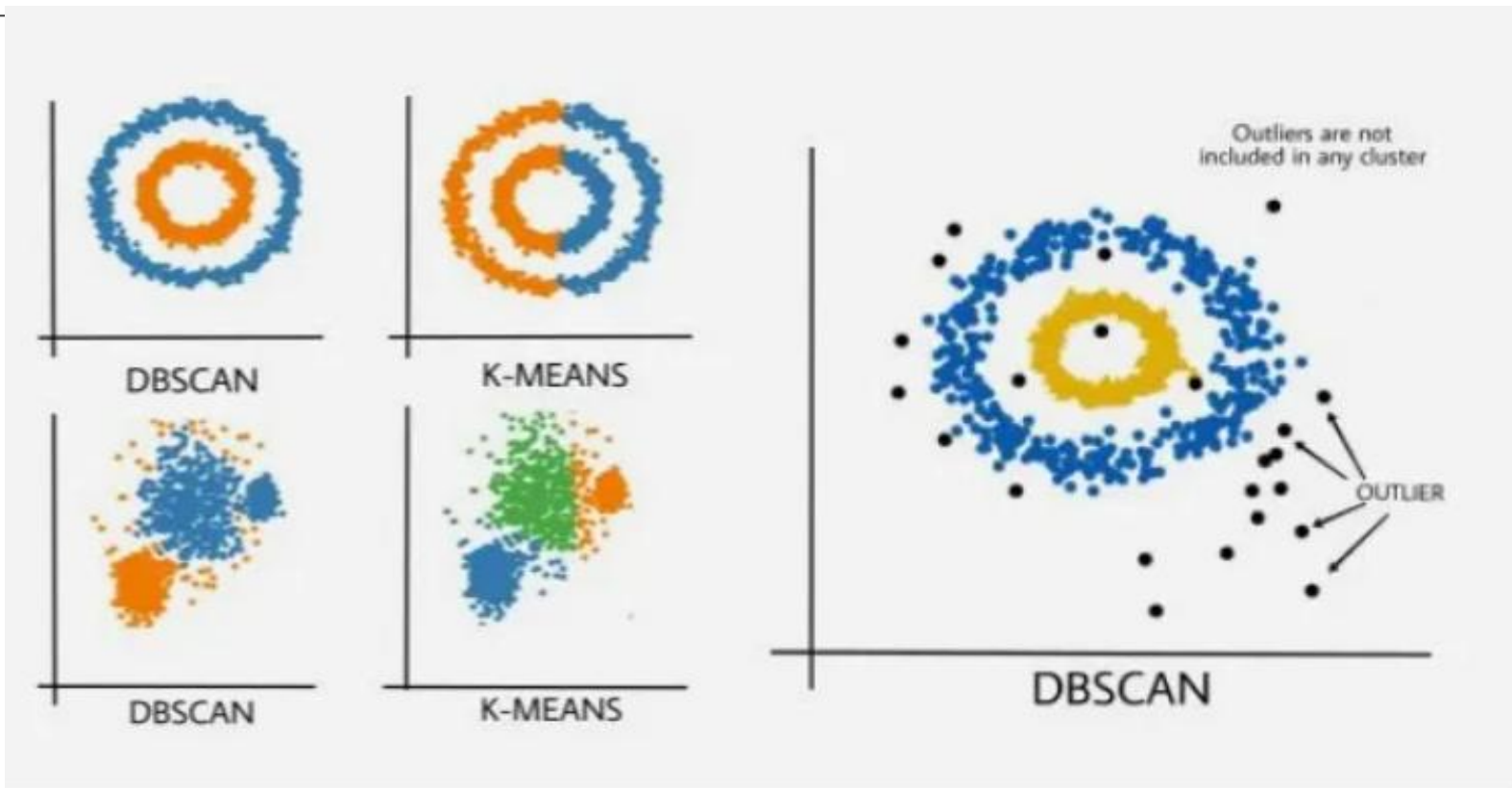
- **Step 4** — Leave all the noise points as it is.



DBSCAN Clustering



DBSCAN vs k means Clustering



DBSCAN Vs K-Means

As it can identify clusters of arbitrary shapes and effectively handle noise. K-Means on the other hand is better suited for data with well-defined, spherical clusters and is less effective with noise or complex cluster structures.

DBSCAN vs k means Clustering

Feature	DBSCAN	K-Means
Clustering Approach	Density-based	Centroid-based
Shape of Clusters	Arbitrary shapes	Spherical
Number of Clusters	Not specified; determined from data	Must be specified (K)
Handling Noise/Outliers	Identifies and handles noise	Does not handle noise explicitly
Scalability	Less scalable for large datasets	<u>Generally</u> more scalable
Parameter Sensitivity	Sensitive to ϵ and MinPts	Sensitive to initialization and K

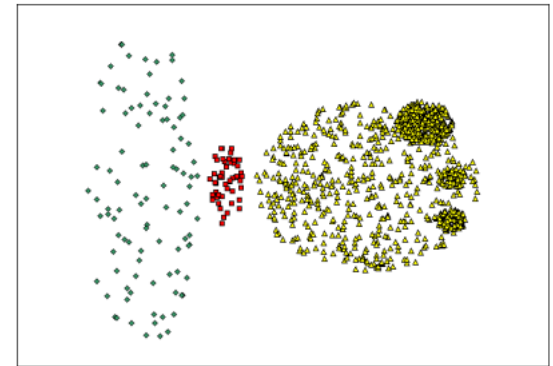
Advantages of DBSCAN

- **Robust to outliers** : It is robust to outliers as it defines clusters based on dense regions of data, and isolated points are treated as noise.
- **No need to specify clusters** : Unlike some clustering algorithms, DBSCAN does not require the user to specify the number of clusters beforehand, making it more flexible and applicable to a variety of datasets.
- **Can find arbitrary shaped clusters** : DBSCAN can identify clusters with complex shapes and is not constrained by assumptions of cluster shapes, making it suitable for data with irregular structures.
- **Only 2 hyperparameters to tune** : DBSCAN has only two primary hyperparameters to tune: “eps” (distance threshold for defining neighborhood) and “min_samples” (minimum number of points required to form a dense region). This simplicity can make parameter tuning more straightforward.

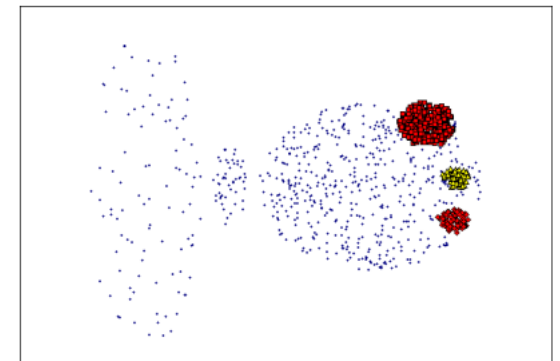


Disadvantages of DBSCAN :

- **Sensitivity to hyperparameters** : The performance of DBSCAN can be sensitive to the choice of its hyperparameters, especially the distance threshold (eps) and the minimum number of points (min_samples). Suboptimal parameter selection may lead to under-segmentation or over-segmentation.



(MinPts=4, Eps=9.75).



(MinPts=4, Eps=7.92)

Disadvantages of DBSCAN :



- **Difficulty with varying density clusters :** DBSCAN struggles with clusters of varying densities. It may fail to connect regions with lower point density to the rest of the cluster, leading to suboptimal cluster assignments in datasets with regions of varying densities.
- **3. Does not predict :** Unlike some clustering algorithms, DBSCAN does not predict the cluster membership of new, unseen data points. Once the model is trained, it is applied to the existing dataset without the ability to generalize to new observations outside the training set.

Application Areas of DBSCAN :

- **Spatial Data Analysis:** DBSCAN is particularly well-suited for spatial data clustering due to its ability to find clusters of arbitrary shapes, which is common in geographic data. It's used in applications like identifying regions of similar land use in satellite images or grouping locations with similar activities in **GIS (Geographic Information Systems)**.
- **Anomaly Detection:** The algorithm's effectiveness in distinguishing noise or outliers from core clusters makes it useful in anomaly detection tasks, such as detecting fraudulent activities in banking transactions or identifying unusual patterns in network traffic.
- **Customer Segmentation:** In marketing and business analytics, DBSCAN can be used for customer segmentation by identifying clusters of customers with similar buying behaviors or preferences.
- **Environmental Studies:** DBSCAN can be used in environmental monitoring, for example, to cluster areas based on pollution levels or to identify regions with similar environmental characteristics.
- **Traffic Analysis:** In traffic and transportation studies, DBSCAN is useful for identifying hotspots of traffic congestion or for clustering routes with similar traffic patterns.
- **Machine Learning and Data Mining:** More broadly, in the fields of machine learning and data mining, DBSCAN is employed for exploratory data analysis, helping to uncover natural structures or patterns in data that might not be apparent otherwise.



DBSCAN Numerical

- Apply the DBSCAN algorithm to the given data points and
- Create the clusters with
- $\text{minPts} = 4$ and
- $\text{epsilon} (\epsilon) = 1.9$.

Data Points:

P1: (3, 7)	P2: (4, 6)
P3: (5, 5)	P4: (6, 4)
P5: (7, 3)	P6: (6, 2)
P7: (7, 2)	P8: (8, 4)
P9: (3, 3)	P10: (2, 6)
P11: (3, 5)	P12: (2, 4)



DBSCAN Numerical

minPts = 4 and epsilon (ϵ) = 1.9

P1: (3, 7)

P2: (4, 6)

P3: (5, 5)

P4: (6, 4)

P5: (7, 3)

P6: (6, 2)

P7: (7, 2)

P8: (8, 4)

P9: (3, 3)

P10: (2, 6)

P11: (3, 5)

P12: (2, 4)

	P1	P2	P3	P4	P5	P6	P7	P8	P9	P10	P11	P12
P1	0											
P2	1.41	0										
P3	2.83	1.41	0									
P4	4.24	2.83	1.41	0								
P5	5.66	4.24	2.83	1.41	0							
P6	5.83	4.47	3.16	2.00	1.41	0						
P7	6.40	5.00	3.61	2.24	1.00	1.00	0					
P8	5.83	4.47	3.16	2.00	1.41	2.83	2.24	0				
P9	4.00	3.16	2.83	3.16	4.00	3.16	4.12	5.10	0			
P10	1.41	2.00	3.16	4.47	5.83	5.66	6.40	6.32	3.16	0		
P11	2.00	1.41	2.00	3.16	4.47	4.24	5.00	5.10	2.00	1.41	0	
P12	3.16	2.83	3.16	4.00	5.10	4.47	5.39	6.00	1.41	2.00	1.41	0



DBSCAN Numerical

P1: P2, P10

P2: P1, P3, P11

P3: P2, P4

P4: P3, P5

P5: P4, P6, P7, P8

P6: P5, P7

P7: P5, P6

P8: P5

P9: P12

P10: P1, P11

P11: P2, P10, P12

P12: P9, P11

minPts = 4 and epsilon (ϵ) = 1.9

Point	Status	
P1	Noise	Border
P2	Core	
P3	Noise	Border
P4	Noise	Border
P5	Core	
P6	Noise	Border
P7	Noise	Border
P8	Noise	Border
P9	Noise	
P10	Noise	Border
P11	Core	
P12	Noise	Border



minPts = 4 and **epsilon (ϵ) = 1.9**

P1: P2, P10

P2: P1, P3, P11

P3: P2, P4

P4: P3, P5

P5: P4, P6, P7, P8

P6: P5, P7

P7: P5, P6

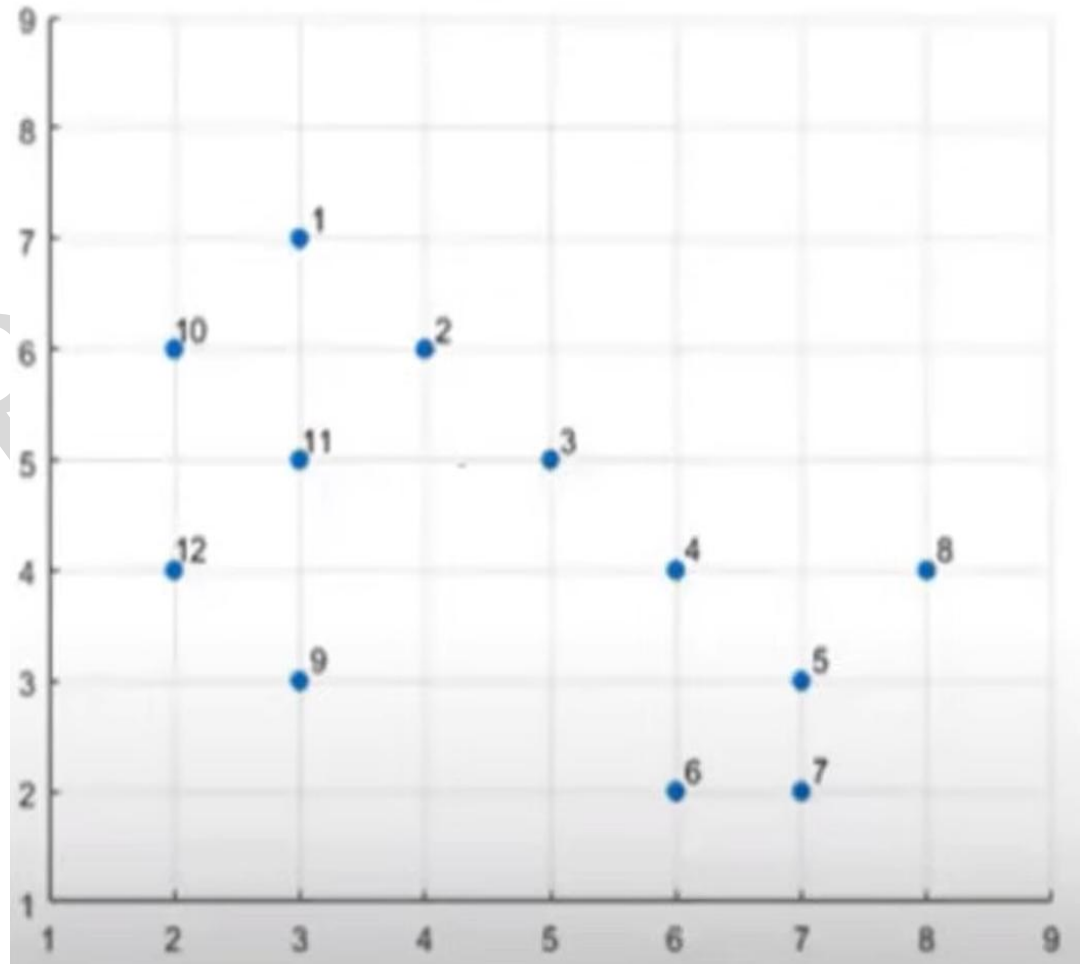
P8: P5

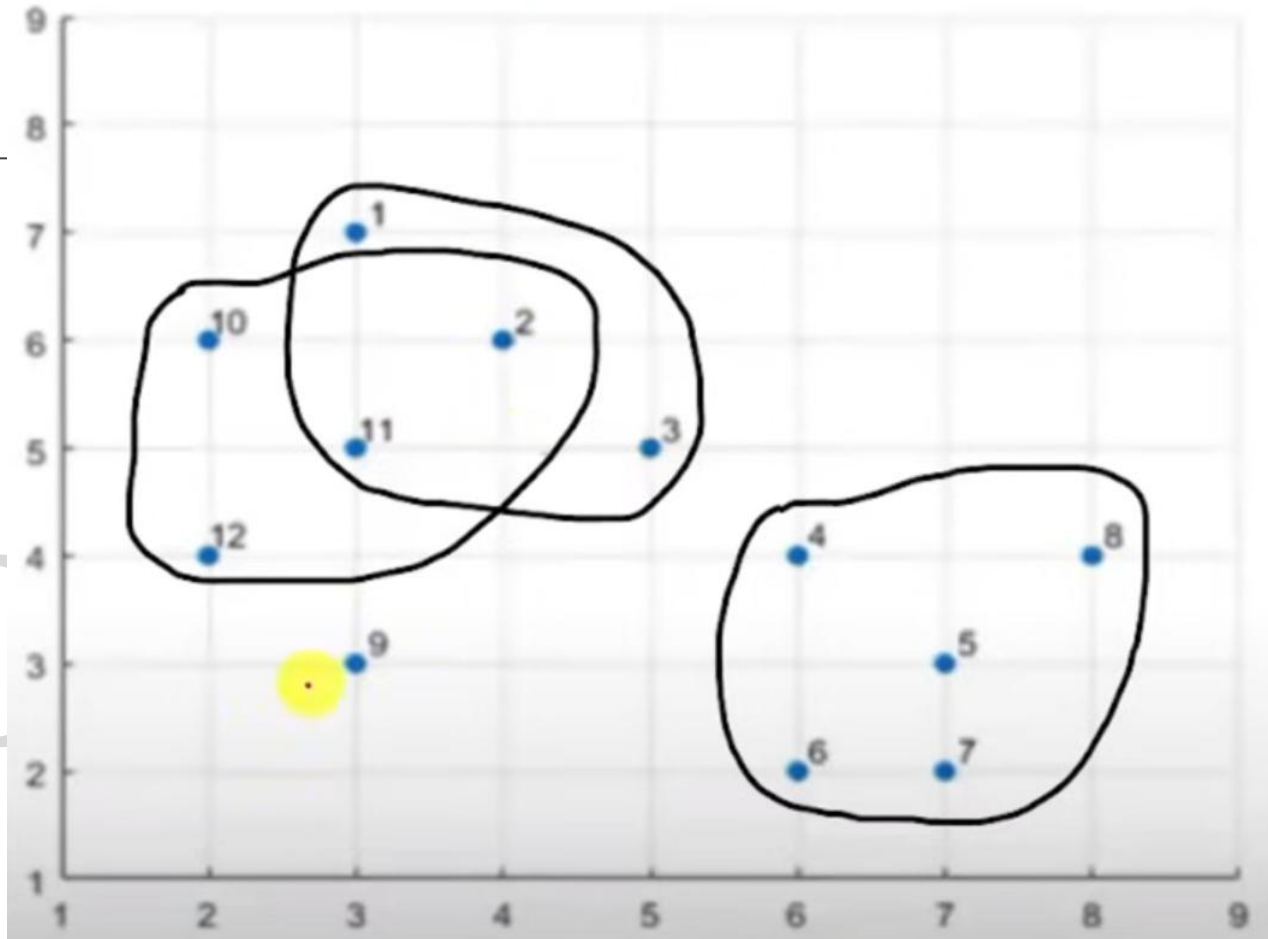
P9: P12

P10: P1, P11

P11: P2, P10, P12

P12: P9, P11





There will be two clusters. In **DBSCAN**, clusters are formed based on **density connectivity**:

- Even if you find multiple **core points**, they **do NOT** form separate clusters if they are **density-reachable** from each other.



Thank You



D Y PATIL
UNIVERSITY

NAVI MUMBAI